

IntroSec_T1_zsmf

Disclaimer

Bei der Erstellung der folgenden Inhalte wurde viel mit LLMs gearbeitet. Keine Gewähr auf Richtigkeit.

Inhalt:

- 01_Computer_Security
- 02_Symmetric_Cryptography
- 03_Asymmetric_Cryptography
- 04_PKI_and_Crypto_Protocols
- 05_Authentication
- 06_Access_Control_and_Containers
- 07_Network_Security

01_Computer_Security

Computer Security - Einführung & Konzepte

Was ist Computer Security?

Prävention von Schäden an Computern und Kommunikationssystemen, deren Schutz und Wiederherstellung.

Das Ziel ist es, **Verfügbarkeit**, **Integrität**, **Authentizität**, **Vertraulichkeit** und **Nicht-Abstreitbarkeit** (Nonrepudiation) sicherzustellen (nach NIST).

Zentrale Sicherheitskonzepte (Key Security Concepts)

- **Vertraulichkeit (Confidentiality):** Sensible Daten werden nicht an unbefugte Entitäten weitergegeben.
- **Integrität (Integrity):**
 - *Daten:* Wurden nicht unbefugt oder unbemerkt verändert.
 - *System:* Führt seine Funktion unbeeinträchtigt aus.
- **Verfügbarkeit (Availability):**
 - *Daten:* Berechtigte Benutzer haben Zugriff auf gespeicherte Daten.
 - *System:* Arbeitet prompt, der Dienst wird berechtigten Nutzern nicht verweigert.
- **Authentizität (Authenticity):** Daten stammen von ihrer vorgeblichen Quelle.
- **Zurechenbarkeit (Accountability):** Aktionen können eindeutig einer bestimmten Entität zugeordnet werden.
- **Nicht-Abstreitbarkeit (Nonrepudiation):** Die Unfähigkeit, die Verantwortung für eine bestimmte Handlung abzustreiten.

Security ist komplex

Sicherheitseigenschaften sind im Allgemeinen **unentscheidbar**!

Während normale Funktionalität erwartet, dass bei *erwartetem Input* eine *erwartete Aktion* folgt, muss Security sicherstellen, dass bei *unerwarteten (böartigen) Aktionen* **keine** schädlichen Dinge passieren (was bei potenziell unendlich vielen Inputs schwer zu garantieren ist).

Schwachstellen, Bedrohungen & Angriffe (Vulnerabilities, Threats & Attacks)

- **Vulnerabilities (Schwachstellen):** Schwächen in Systemen, Prozeduren oder Implementierungen, die von Angreifern ausgenutzt werden können.

- **Beispiele:** Schwache Passwörter, Code Injection, fehlerhafte Speicherverwaltung (Memory safety), unsichere Kryptographie.
- **Threat (Bedrohung):** Ein Umstand oder Ereignis, das potenziell die Sicherheit gefährden kann.
 - **Intern:** Akteure **mit** autorisiertem Zugriff.
 - **Extern:** Akteure **ohne** autorisierten Zugriff.
- **Attack (Angriff):** Ein konkreter Versuch, Schwachstellen auszunutzen.
 - **Passiv:** Versuch, an Informationen durch bloßes Beobachten zu gelangen (z.B. Sniffing).
 - **Aktiv:** Versuch, Systemressourcen zu verändern oder ihren Betrieb zu stören.

🔗 Rechtliche Lage in Österreich (Strafgesetzbuch - StGB)

Angriffe auf reale Systeme ohne Zustimmung sind **illegal!**

- §118a: Widerrechtlicher Zugriff auf ein Computersystem
- §119, §119a: Abfangen von Daten
- §126a, §126b, §126c: Datenbeschädigung, Störung von IT-Systemen, Missbrauch von Programmen
- **Strafen:** Bis zu 360 Tagessätze oder bis zu 5 Jahre Haft.

Kurze Geschichte des Hackings

- **1903 (Erster "Hack"):** N. Maskelyne fälschte Nachrichten bei der Demo von Marconis "sicherem" drahtlosen Telegraphen.
- **1938-1945 (Kryptobomben):** Entschlüsselung der Enigma (Polen & Alan Turing).
- **1960er-70er (Phreaking):** Manipulation des Telefonnetzes. Kostenlose Ferngespräche durch Senden von Tönen (2600 Hz), die das Ende eines Anrufs simulierten, aber die Leitung offen hielten.
- **1970er (Erste Malware):** **Creeper** (1971, erster Wurm, harmlos) und **Reaper** (1972, erstes Anti-Malware-Programm, löschte Creeper).
- **1980er-2000er (Steigende Kosten):** * **Morris Worm (1988):** Erste große mediale Aufmerksamkeit, Millionen-Schäden.
 - **AIDS (1989):** Erste bekannte Ransomware.
 - **ILOVEYOU (2000):** E-Mail-Wurm (VBScript), >10 Mio. infizierte PCs in wenigen Tagen, \$15 Mrd. Schaden.

Die heutige Sicherheitslandschaft (Current Landscape)

- Cyber-Unsicherheit ist weltweit das 9. größte Risiko.
- Geschätzte Kosten von Cyberkriminalität: 10,8 Billionen \$ (2026).

- Es gibt einen extremen Mangel an Sicherheitsexperten.

Kommerzialisierung von Schwachstellen

- **Bug Bounty Programme:** Entwickler zahlen legal Belohnungen an Sicherheitsforscher (White-Hats) für gefundene Lücken (z.B. Google, Apple zahlen bis zu Millionenbeträge).
- **Gray Markets:** Exploits werden an Regierungen / Geheimdienste verkauft.
- **Black Markets:** Verkauf von Exploits im Dark Web gegen Kryptowährungen. Höhere Auszahlungen, aber illegal und gefährlich.

Bedrohungsakteure (Threat Actors)

1. **Cybercriminals:** Infiltrieren Systeme für Geld (z.B. durch DDoS-for-Hire oder Botnets).
2. **Ideologues (Hacktivisten/Terroristen):** Politisch/sozial motiviert; wollen Aufmerksamkeit erregen oder Schrecken verbreiten.
3. **Nation-state actors:** Staatliche Akteure (z.B. *Stuxnet* gegen iranische Atomanlagen, *Operation Aurora* für IP-Diebstahl). Ziel: Spionage, Sabotage.
4. **Insiders:** Interne Mitarbeiter, die ihr System angreifen (schwer abzuwehren, da sie bereits Teilzugriff haben).

Ethisches Hacking & Vulnerability Management

Ethisches Hacking (White-Hat)

Identifizieren von Schwachstellen **mit** Zustimmung des Eigentümers. Verursacht keinen Schaden und meldet Fehler verantwortungsvoll.

- **Do's & Don'ts:** Auf eigenen/Test-Systemen experimentieren, versehentlich geleakte Nutzerdaten sofort löschen, keine destruktiven Angriffe durchführen!
- **Responsible Disclosure (Verantwortungsvolle Offenlegung):** Forscher melden Lücken privat. Der Entwickler hat eine Deadline (meist 90-120 Tage), um einen Patch zu veröffentlichen, bevor die Lücke publiziert wird. Das gibt Zeit für Fixes.

CVE und CVSS

- **CVE (Common Vulnerabilities and Exposures):** Eindeutige IDs für öffentliche Schwachstellen (Format: `CVE-YYYY-ID`).
 - Ausgestellt von **CNAs** (CVE Numbering Authorities, z.B. MITRE, Oracle).
- **CVSS (Common Vulnerability Scoring System):** Bewertet, wie schlimm/kritisch eine Schwachstelle ist (Score 0.0 - 10.0).

Eine kritische Schwachstelle in OpenSSL. Durch einen Puffer-Überlesefehler (Buffer Over-read) bei der "Heartbeat"-Erweiterung konnten Angreifer private Keys aus dem Speicher des Servers auslesen. CVSS Score: 8.7 (High).



Nutzer sind oft das schwächste Glied im Sicherheitssystem ("*Human Error*"). Selbst bei bester Kryptographie und Firewalls können Phishing, schwache Passwörter (Passwort-Zettel am Monitor) oder Unachtsamkeit (z.B. versehentliches Einspeisen vertraulicher Daten in öffentliche KIs) zur Kompromittierung führen.

Security Principles (Sicherheitsprinzipien)

Grundsätze für das Design sicherer Systeme:

1. **Simplicity (KISS - Keep it simple stupid):** Einfache Systeme sind leichter zu verstehen, zu prüfen und enthalten seltener Fehler. (Occam's razor).
2. **Open Design:** Die Sicherheit darf *nicht* auf der Geheimhaltung des Mechanismus beruhen ("Security through obscurity" ist schlecht!). Sie darf nur von der Geheimhaltung des *Schlüssels/Passworts* abhängen (Kerckhoffs-Prinzip).
3. **Compartmentalization (Isolierung):** Aufteilung von Ressourcen in isolierte Gruppen (z.B. User vs. Kernel Space, Virtuelle Maschinen). Verhindert, dass sich Fehler/Angriffe auf das ganze System ausbreiten.
4. **Minimum Exposure:** Angriffsfläche minimieren (Attack Surface). Nur absolut notwendige Dienste nach außen freigeben und Fehlermeldungen informationsarm halten (z.B. keine

Softwareversionen bei Errors anzeigen).

5. **Least Privilege (Minimales Privileg):** Jeder Benutzer/Prozess sollte nur genau die Rechte haben, die er für seine aktuelle Aufgabe benötigt (verhindert u.a. *Confused Deputy Attacks*).
6. **Minimum Trust, Maximum Trustworthiness:** Vertraue nichts und niemandem (z.B. "Never trust user input"). Ersetze blindes Vertrauen durch bewiesene/validierte Eigenschaften (z.B. immer Input filtern/validieren).
7. **Fail-Safe Defaults:** Bei einem Ausfall muss das System in einen sicheren Zustand zurückfallen. Bei Access Control bedeutet das *Allowlisting*: Zugriff ist standardmäßig verboten, es sei denn, er wird explizit erlaubt.
8. **Complete Mediation:** Jeder Zugriff auf ein sicherheitsrelevantes Objekt muss unumgebar überwacht und kontrolliert werden. Um Angriffe auf tiefere Ebenen (z.B. USB-Boot) zu verhindern, wird Verschlüsselung genutzt.
9. **Defense in Depth (Tiefenverteidigung):** Redundante (mehrere) Sicherheitsschichten einbauen (z.B. PIN + Karte = 2FA). Fällt eine Schicht, greift die nächste.
10. **Traceability (Nachvollziehbarkeit):** Alle sicherheitsrelevanten Events müssen protokolliert (geloggt) werden, um Aktionen jemandem zurechnen zu können (Accountability). *Wichtig:* Logs müssen vor Manipulation geschützt werden.
11. **Generating secure secrets (Entropie maximieren):** Geheimnisse (Keys, Passwörter) müssen maximal zufällig/unvorhersehbar sein (z.B. Shannon-Entropie), um Brute-Force- oder Wörterbuchangriffe zu erschweren.
12. **Usability (Benutzerfreundlichkeit):** Security-Mechanismen müssen einfach zu bedienen sein und dürfen den Workflow nicht behindern. Sonst machen Nutzer Fehler oder schalten die Security einfach ab!

02_Symmetric_Cryptography

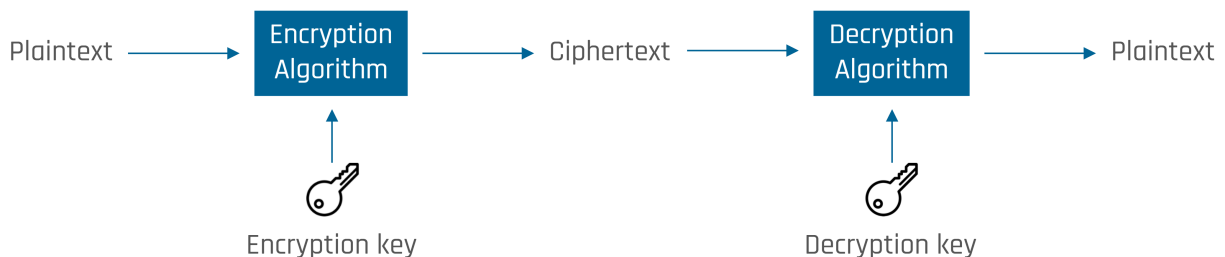
Einführung in die Kryptographie

Definitionen der Kryptographie

Klassisch: Die Wissenschaft von Techniken zur Verschlüsselung (Encryption) und Entschlüsselung (Decryption) von Daten.

Modern: Die Praxis und das Studium von Techniken für sichere Kommunikation in der Präsenz von feindlichem Verhalten (Adversaries). Dies umfasst auch Hash-Funktionen, Message Authentication Codes (MACs) und digitale Signaturen.

Cryptography is the science of techniques for encrypting and decrypting data



Symmetrische vs. Asymmetrische Kryptographie

- **Symmetrische Kryptographie:**
 - Verwendet den **gleichen Schlüssel** für kryptographische Operationen (Verschlüsselung & Entschlüsselung).
 - Der Schlüssel muss geheim bleiben und sicher übertragen werden.
 - Jedes Kommunikationspaar benötigt einen anderen Schlüssel.
 - Sehr gute Performance (schnell).
- **Asymmetrische Kryptographie:**
 - Verwendet **unterschiedliche Schlüssel** (einen zum Verschlüsseln, einen anderen zum Entschlüsseln).
 - Nur ein Schlüssel bleibt geheim (Private Key), der andere ist öffentlich verfügbar (Public Key).
 - Eher langsam.

Klassische Chiffren (Classical Ciphers)

Grundlegende Transformationen

Obwohl klassische Chiffren heute unsicher sind, bilden sie die Basis (Substitution und Transposition) für moderne symmetrische Chiffren.

- **Substitution:** Symbole des Plaintexts werden durch andere Symbole ersetzt.
 - *Simple vs. polygraphic:* Ersetzt ein oder mehrere Symbole auf einmal.
 - *Monoalphabetisch vs. polyalphabetisch:* Eine oder mehrere mögliche Ersetzungen, abhängig von der Position im Text.
- **Transposition:** Symbole bleiben erhalten, aber ihre Reihenfolge wird vertauscht (Permutation).

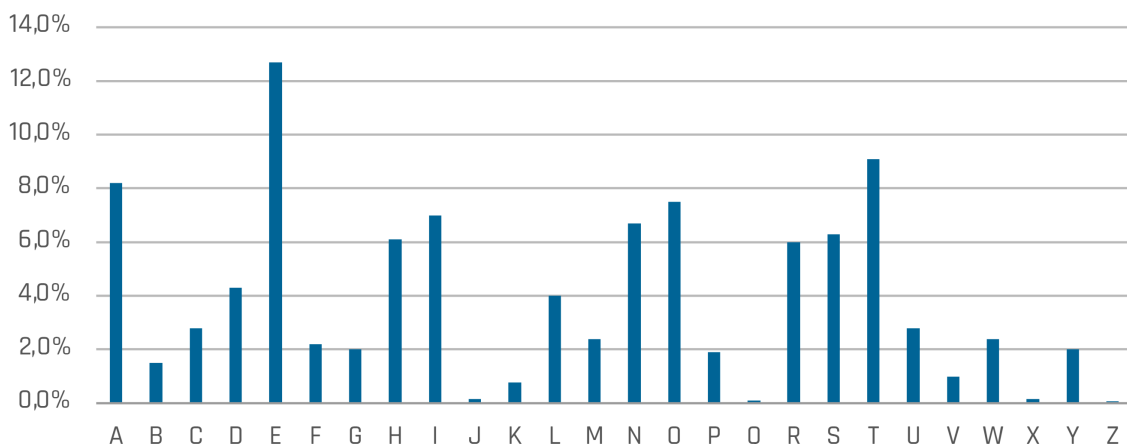
Shift Cipher (Caesar Chiffre)

- Jeder Buchstabe wird um eine **festen Anzahl an Positionen** im Alphabet verschoben. Diese Zahl ist der Schlüssel.
- **Sicherheit:** Sehr schwach. Bei 26 Buchstaben gibt es nur 25 mögliche Schlüssel. Ein **Brute-Force-Angriff** (Ausprobieren aller Schlüssel) ist trivial.

Einfache Monoalphabetische Substitution

- Jedes Symbol wird auf ein (möglicherweise anderes) Symbol abgebildet (z.B. A->Q, B->M, ...).
- **Sicherheit:** Ein Brute-Force-Angriff ist zeitlich unmöglich (26! mögliche Schlüssel, dauert Milliarden Jahre). Dennoch ist es **nicht sicher**!
- **Statistische Kryptoanalyse:** Da Buchstaben in natürlichen Sprachen unterschiedlich oft vorkommen (z.B. "E" sehr häufig), bleibt diese Häufigkeitsverteilung im Ciphertext erhalten. Durch Auszählen der Buchstabenhäufigkeiten und Raten von Mustern (z.B. kurze Wörter wie "THE" oder "A") lässt sich der Text brechen.

Letters Frequencies in English



Vigenère Cipher (Polyalphabetisch)

- Verbirgt die Häufigkeiten, indem Buchstaben je nach Position unterschiedlich verschoben werden.
- Der Schlüssel ist eine Folge von Zahlen (bzw. Buchstaben), die wiederholt wird.
- **Sicherheit:** Brute-Force meist unmöglich (zu viele Schlüsselmöglichkeiten 26^l). Statistische Analysen sind aber effektiv bei längeren Texten:
 1. **Schlüssellänge (l) bestimmen:** Mit Kasiski-Test (sucht nach wiederkehrenden Mustern) oder Friedman-Test (nutzt den Koinzidenzindex).
 2. Text in Gruppen aufteilen (jede Gruppe entspricht einem Shift Cipher) und per Häufigkeitsanalyse brechen.

Columnar Transposition (Spaltentransposition)

- Text wird zeilenweise in eine Matrix geschrieben und spaltenweise basierend auf dem Schlüssel (Zahlenfolge, z.B. 3, 1, 2, 5, 4) ausgelesen.
- **Sicherheit:** Brute-Force nur für kurze Schlüssel praktikabel. Für längere Schlüssel nutzt man heuristische Optimierungen wie "Hill climbing".

Formale Definitionen & Perfect Secrecy

Symmetrisches Verschlüsselungsschema

Ein Tripel aus Algorithmen (Gen, Enc, Dec) mit Key Space $\mathcal{K}$, Message Space $\mathcal{M}$ und Ciphertext Space $\mathcal{C}$:

- Gen : Generiert einen zufälligen Schlüssel $k \in \mathcal{K}$.
- Enc : Nimmt $k \in \mathcal{K}$ und Nachricht $m \in \mathcal{M}$ und erzeugt Ciphertext $c \in \mathcal{C}$.
- Dec : Nimmt $k \in \mathcal{K}$ und $c \in \mathcal{C}$ und stellt $m \in \mathcal{M}$ wieder her.

Korrektheit: $Dec(k, Enc(k, m)) = m$.

Perfect Secrecy (Perfekte Sicherheit)

- **Informations-theoretische Definition (Claude Shannon):** Die Beobachtung des Ciphertexts verrät dem Angreifer **keinerlei neue Informationen** über den Plaintext.
- **Formal:** $Pr[M = m | C = c] = Pr[M = m]$. (Die Wahrscheinlichkeit, dass die Nachricht m ist, ändert sich nicht, selbst wenn wir c sehen).
- **Voraussetzung:** Ein perfektes Chiffre benötigt mindestens genauso viele Schlüssel wie es Nachrichten gibt: $|\mathcal{K}| \geq |\mathcal{M}|$.

The One-Time-Pad (OTP)

- Erstes nachgewiesen perfekt sicheres Chiffre.
- Schlüssel ist exakt **genauso lang wie der Plaintext** und zufällig generiert.
- Operation: **Bitweises XOR** ($Enc(k, m) = k \oplus m$ und $Dec(k, c) = k \oplus c$).

⚠ Probleme des One-Time-Pads

- Schlüsselverteilung und Speicherung sind extrem unpraktisch (Schlüssel so groß wie die Datei).
- Schlüssel darf **nur ein einziges Mal** verwendet werden!
- Bei Wiederverwendung bricht die Sicherheit komplett: $c_1 \oplus c_2 = m_1 \oplus m_2$ (die Schlüssel heben sich auf!).
- **Keine Integrität:** Chiffre ist **malleable** (veränderbar).

Moderne Kryptographie

Produktchiffren (Product Ciphers)

- Kombinieren mehrere Transformationen iterativ (meist Substitution, Transposition und modulare Arithmetik), um stärker zu sein als die Einzelteile.
- ⚠ **Wichtig: Nicht alle Kombinationen sind sinnvoll. Die Kombination von zwei monoalphabetischen Substitutionen ergibt nur wieder eine einfache monoalphabetische Substitution.**

📄 Confusion & Diffusion (Shannon)

- **Confusion:** Verschleiert die Beziehung zwischen Plaintext und Ciphertext. Änderungen am Plaintext sollen unvorhersehbare Auswirkungen auf den Ciphertext haben.
- **Diffusion:** Jedes Bit des Plaintexts/Schlüssels soll über den gesamten Ciphertext verteilt werden. Ein gekipptes Bit im Plaintext sollte im Schnitt die Hälfte der Ciphertext-Bits ändern (**Avalanche Effect**).

Stromchiffren vs. Blockchiffren

Block Ciphers

100001
010...

Stream Ciphers

- Plaintext is split into blocks of **equal length**
- **Padding algorithms** are used to fill the last block if shorter than the block length
- A **mode of operation** determines how the encryption is applied to each block
- The same key is used for the encryption of every block
- A sequence of pseudorandom bytes (**keystream**) is derived from the **key**
 - Sometimes already processed parts of plaintext or ciphertext are used
- The keystream is combined with the plaintext (typically via **bitwise XOR**) to produce the ciphertext

Stromchiffren (Stream Ciphers)

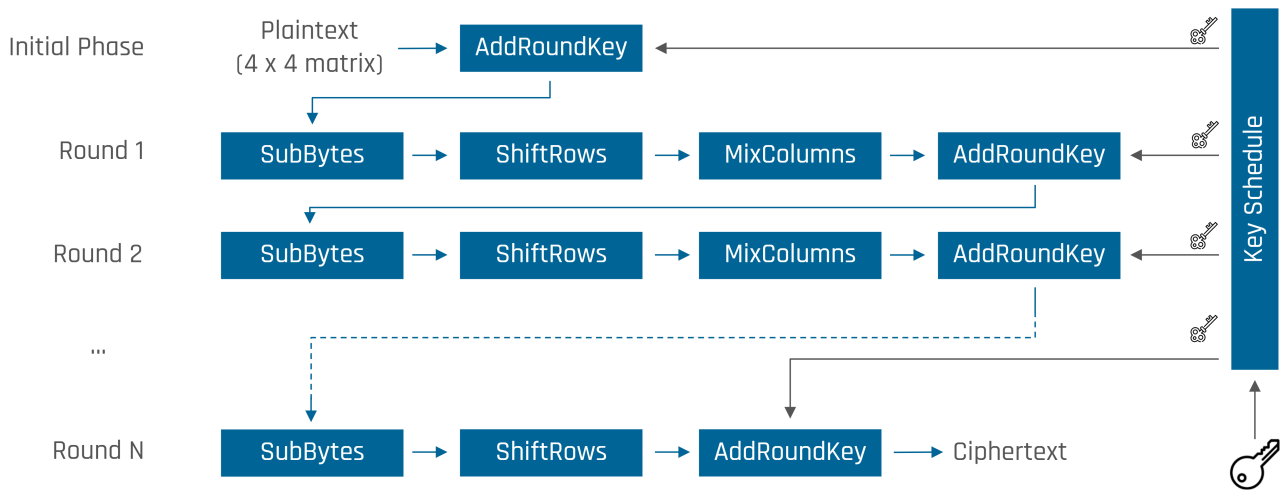
- Annäherung an das One-Time-Pad.
- Der Schlüssel dient als Seed für einen **Pseudorandom Generator (PRG)**, der einen endlosen *Keystream* generiert.
- Der Keystream wird via XOR mit dem Plaintext verknüpft.
- *Synchron vs. Self-synchronizing*: Synchron (Status hängt nur vom vorherigen PRG-Status ab) vs. Self-sync (Status hängt von vorherigen Texten ab).
- **Sicherheit**: Keine Perfect Secrecy, hängt rein vom PRG ab.
- **Malleability (Veränderbarkeit)**: Stromchiffren sind stark malleable! Wenn ein Angreifer c ändert in $c \oplus t$, entschlüsselt sich das zu $m \oplus t$. Der Angreifer kann also gezielt Bits kippen.
- *Beispiel: RC4* (Entwickelt 1984, generiert Permutationen basierend auf zwei Indizes. Ist heutzutage gebrochen!).

Blockchiffren (Block Ciphers) & AES

- Plaintext wird in Blöcke fester Länge zerlegt.
- Gleicher Schlüssel für alle Blöcke.
- Padding füllt den letzten Block auf.

AES (Advanced Encryption Standard)

- Standard-Blockchiffre, basiert auf dem *Rijndael*-Algorithmus.
- **Blockgröße**: 128 Bits.
- **Schlüssellängen**: 128 Bit (10 Runden), 192 Bit (12 Runden) oder 256 Bit (14 Runden).
- Besteht aus mehreren Runden mit folgenden Schritten:
 1. **SubBytes**: Ersetzt Bytes basierend auf einer festen S-Box. Bringt **Non-Linearity** (Confusion).
 2. **ShiftRows**: Verschiebt Zeilen der Matrix zyklisch. Sorgt dafür, dass Spalten nicht unabhängig verarbeitet werden (Diffusion).
 3. **MixColumns**: Multipliziert jede Spalte mit einer festen Matrix. Jedes Input-Byte beeinflusst alle Output-Bytes (starke Diffusion).
 4. **AddRoundKey**: Der aktuelle State wird mit dem Subkey der Runde per XOR verknüpft. (Subkeys werden durch den Key Schedule generiert).



Modes of Operation

Wie verschlüsselt man Nachrichten, die länger als ein Block sind?

(Hier geht es rein um Vertraulichkeit, Authentizität/Integrität ist nicht gegeben!)

1. ECB (Electronic Code Book Mode)

- Blöcke werden unabhängig voneinander verschlüsselt.
- Vorteil: Parallelisierbar, Random Read Access.

⚡ Problem

Identische Plaintext-Blöcke ergeben identische Ciphertext-Blöcke. **Muster im Text bleiben sichtbar** (z.B. bei Bildern). Stark Malleable (Angreifer kann Blöcke neu zusammensetzen). **Sollte niemals verwendet werden!**

2. CBC (Cipher Block Chaining Mode)

- Jeder Block wird vor der Verschlüsselung mit dem **vorherigen Ciphertext-Block XOR-verknüpft**.
- Der erste Block nutzt einen zufälligen **Initialization Vector (IV)**.
- Verbirgt Muster.
- Nachteile: Verschlüsselung ist **nicht** parallelisierbar (muss sequenziell laufen). Entschlüsselung ist parallelisierbar.
- Fehlerfortpflanzung: Ein gekipptes Bit im Ciphertext zerstört den aktuellen Block und kippt das exakt selbe Bit im nächsten Plaintext-Block.

3. CTR (Counter Mode)

- Macht aus der Blockchiffre eine **Stromchiffre**.
- Verschlüsselt wird ein Zähler (*Nonce + Counter*), der Output ist der Keystream, der dann per XOR mit dem Plaintext verknüpft wird.
- Vorteile: Voll parallelisierbar, Random Read Access.
- Nachteil: Nonce darf niemals wiederverwendet werden! Malleable wie Stromchiffren.

Padding (PKCS#7)

Wenn der letzte Block nicht voll ist, werden N Bytes mit dem Wert N angehängt. (z.B. fehlen 4 Bytes → anhängen von `04 04 04 04`). Ist der Block voll, wird ein ganzer Dummy-Block angehängt, der nur die Blockgröße enthält.

Hash Funktionen

- Bilden Eingaben beliebiger Länge auf eine **fixe Ausgabelänge** ab.
- Da die Eingabemenge unendlich, die Ausgabemenge aber endlich ist, sind **Kollisionen mathematisch unvermeidbar** (Pigeonhole Principle).

Eigenschaften Kryptographischer Hash-Funktionen

1. **One-way (Einweg):** Es ist praktisch unmöglich, aus dem Hashwert den ursprünglichen Text zu berechnen.
2. **Uniformity:** Ausgaben wirken komplett zufällig gleichverteilt.
3. **Diffusion:** Eine winzige Änderung im Input verändert den Output fast komplett (Avalanche Effect).
4. **Kollisionsresistenz (Collision Resistance):**
 - **Weak:** Gegeben ist m , finde ein anderes m' , sodass $h(m) = h(m')$.
 - **Strong:** Finde **irgendwelche** zwei m, m' , die den gleichen Hash haben.

Bekannte Hash Algorithmen:

- MD5 (128 Bit) & SHA-1 (160 Bit) → **Gebrochen/Unsicher!**
- SHA-2 & SHA-3 (224 bis 512 Bit) → **Sicher.**

Verwendungszwecke: Integritätsprüfung (File Hashes), Digitale Signaturen (um große Daten schneller zu signieren), Passwort-Speicherung.

Message Authentication Codes (MAC)

- Sichern Integrität und **Authentizität**.
- Nutzen Blockchiffren oder Hash-Funktionen in Kombination mit einem **Shared Key**. Ein Angreifer kann die Nachricht nicht ändern, da er den Key zum Berechnen des neuen MACs nicht kennt.

Konzepte:

- **CBC-MAC:** Verschlüsselt im CBC-Modus mit Null-IV. Der letzte Block ist der MAC. *Achtung:* Nur für Nachrichten fester Länge sicher! (Anfällig für Variable-Length Attacks).
 - **Encrypted CBC-MAC:** Der letzte Block wird mit einem zusätzlichen Key nochmals verschlüsselt. Löst das Längenproblem.
 - **HMAC:** Basiert auf Hashes (h). Vermeidet Length-Extension-Attacks durch ein bestimmtes Schema mit einem innen- und außenliegenden Key-Pad (ipad, opad).
 - Formel: $HMAC(k, m) = h((k' \oplus opad) || h((k' \oplus ipad) || m))$.
-

Authenticated Encryption

Wie kombiniert man Verschlüsselung (Confidentiality) und MAC (Integrity/Authenticity) sicher?

1. Encrypt-then-MAC (Sicher!)

- Text verschlüsseln → MAC über den Ciphertext berechnen.
- Wird z.B. in IPsec genutzt. Benötigt zwei verschiedene Keys!

2. MAC-then-Encrypt (Problematisch)

- MAC über Plaintext berechnen → beides zusammen verschlüsseln.
- Früher in SSL/TLS genutzt, führte zu massiven *Padding-Oracle Attacks* (wie POODLE).

3. Encrypt-and-MAC (Problematisch)

- Plaintext verschlüsseln UND separat MAC über Plaintext berechnen.
- Genutzt in SSH. Nicht pauschal sicher.

✓ Best Practice: Authenticated Encryption Modes

Heutzutage nutzt man standardisierte Modi, die beides gleichzeitig (Verschlüsselung + Authentifizierung) von Haus aus sicher machen, z.B. **GCM, CCM, EAX**. In TLS 1.3 sind nur noch diese Modi zugelassen.

03_Asymmetric_Cryptography

Key Management (Schlüsselverwaltung)

Symmetrisch vs. Asymmetrisch

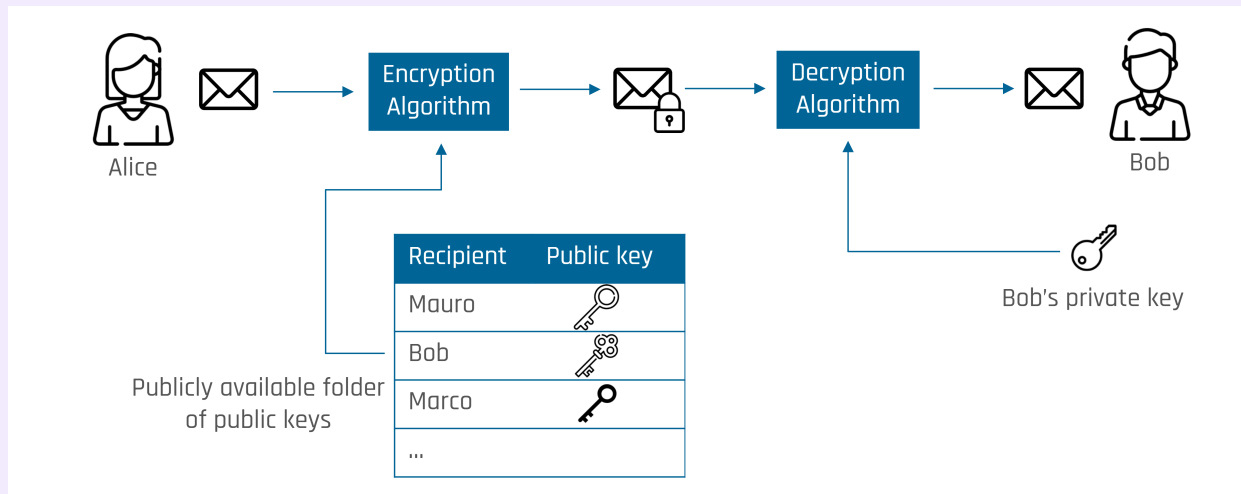
Der Hauptunterschied in der Schlüsselverwaltung liegt in der Skalierbarkeit und der Art der Schlüsselverteilung.

- **Symmetrische Kryptographie:**
 - Für N Kommunikationspartner werden $N \cdot (N - 1)/2$ geheime Schlüssel benötigt, um untereinander zu kommunizieren.
 - Jeder Partner braucht $N - 1$ Schlüssel, die **sicher** (geheim) geteilt werden müssen.
 - Dieser sichere Austausch muss bei jeder Schlüsselerneuerung (Refresh) wiederholt werden.
- **Asymmetrische Kryptographie:**
 - Für N Kommunikationspartner werden nur N Schlüsselpaare benötigt.
 - Jeder Partner hat sein **eigenes Schlüsselpaar** und benötigt die **öffentlichen Schlüssel** der anderen.
 - **Vorteil:** Öffentliche Schlüssel können über unsichere Kanäle (wie das Internet) verteilt werden.

Asymmetrische Verschlüsselung

- Asymmetrische Verschlüsselungsverfahren nutzen **unterschiedliche Schlüssel** für das Verschlüsseln und Entschlüsseln.
- **Wichtigste Regel:** Der Entschlüsselungsschlüssel darf **nicht** aus dem Verschlüsselungsschlüssel ableitbar sein.
 - Das muss selbst dann gelten, wenn ein Angreifer Paare aus Klartexten und dazugehörigen Geheimtexten besitzt (*Known-Plaintext-Angriff*).
- Daher muss der Verschlüsselungsschlüssel nicht geheim gehalten werden.
 - **Verschlüsselungsschlüssel = Public Key** (öffentlicher Schlüssel).
 - **Entschlüsselungsschlüssel = Private Key** (privater Schlüssel).

Ablauf einer asymmetrischen Verschlüsselung



Alice holt sich Bobs Public Key aus einem öffentlichen Verzeichnis. Sie verschlüsselt die Nachricht mit diesem Public Key. Bob entschlüsselt die erhaltene Nachricht mit seinem eigenen Private Key.

Einwegfunktionen (One-Way Functions)

Asymmetrische Verfahren basieren auf speziellen mathematischen Einwegfunktionen, um die Schlüssel-Regel einzuhalten.

- **Integer Factorization (Primfaktorzerlegung):**
 - Das Produkt aus zwei großen Primzahlen lässt sich leicht berechnen.
 - Es ist jedoch praktisch unmöglich (infeasible), eine Zahl aus großen Primzahlen wieder in ihre Faktoren zu zerlegen.
 - *Anwendung:* Die Sicherheit von **RSA** basiert auf diesem Problem.
- **Discrete Logarithms (Diskrete Logarithmen):**
 - Es ist leicht, $g^x \bmod p$ aus g , x und p zu berechnen (selbst bei sehr großen Werten).
 - Es ist praktisch unmöglich, x aus g , p und $g^x \bmod p$ zu berechnen.
 - *Anwendung:* Sicherheit von **DSA**, **ElGamal** und dem **Diffie-Hellman** Protokoll basiert darauf.

Das RSA Verfahren

Hintergrund

Veröffentlicht 1977 von R. Rivest, A. Shamir und L. Adleman. Es kann für Verschlüsselung, digitale Signaturen und Key Management verwendet werden.

Mathematische Grundlagen

- **Eulersche Phi-Funktion $\phi(n)$:** Gibt die Anzahl der natürlichen Zahlen bis n an, die zu n teilerfremd sind (größter gemeinsamer Teiler ist 1).
 - Wenn n prim ist: $\phi(n) = n - 1$.
 - Wenn $n = pq$ (wobei p und q verschiedene Primzahlen sind): $\phi(n) = (p - 1)(q - 1)$.
- **Satz von Euler (Euler's theorem):** Seien a und n teilerfremde, positive Ganzzahlen, dann gilt: $a^{\phi(n)} \equiv 1 \pmod{n}$.

Schlüsselerstellung (Key Generation)

1. Generiere zwei große Primzahlen p und q . *(Größe: Typischerweise 1024 bis 2048 Bit lang. Je größer, desto sicherer, aber auch langsamer)*
2. Berechne den Modulus $N = pq$.
3. Wähle einen öffentlichen Exponenten $e < N$, der zu $\phi(N)$ teilerfremd ist. *(Um die Verschlüsselung zu beschleunigen, ist e meist eine kleine Zahl, z.B. $65537 = 2^{16} + 1$)*
4. Der private Exponent d ist das multiplikative Inverse von e modulo $\phi(N)$.
 - Bedingung: $ed \equiv 1 \pmod{\phi(N)}$.
 - Kann mit dem erweiterten euklidischen Algorithmus berechnet werden.

Ergebnis:

- **Public Key:** (e, N) .
- **Private Key:** (d, N) .

Generierung großer Primzahlen

Generiere zufällig eine ungerade Zahl der gewünschten Länge. Teste, ob kleine Primzahlen (z.B. < 1000) die Zahl teilen. Wende iterativ den **Miller-Rabin-Test** an. Sagt der Test "nicht prim", stimmt das immer. Sagt er "prim", könnte er falsch liegen (Wahrscheinlichkeit $< 1/4$). Eine r -fache Wiederholung senkt die Fehlerwahrscheinlichkeit auf $1/4^r$.

Ver- und Entschlüsselung (Textbook RSA)

Sei $m < N$ der Klartext (Plaintext) und c der Geheimtext (Ciphertext).

- **Verschlüsselung (Encryption):** $c = m^e \bmod N$.
- **Entschlüsselung (Decryption):** $m = c^d \bmod N$.

(Die mathematische Korrektheit basiert darauf, dass $m^{ed} \equiv m \pmod{N}$ für alle positiven Nachrichten $m < N$ gilt, basierend auf dem Satz von Euler.)

Sicherheit und Schwachstellen von RSA

- Die Sicherheit hängt davon ab, wie schwer es ist, den Modulus N zu faktorisieren. Kennt man p und q , kann man d leicht berechnen.

- Die Geheimhaltung von $\phi(N)$ ist extrem wichtig; wird sie geleakt, können p und q leicht berechnet werden.

⚡ Probleme mit "Textbook RSA" (ohne Padding)

- **Determinismus:** Verschlüsselt man dieselbe Nachricht mehrmals mit demselben Public Key, ist der Geheimtext immer identisch (Gefahr durch Raten/Mustererkennung).
- **Kleine Nachrichten & kleine Exponenten:** Wenn m und e klein sind, ist m^e kaum größer als N . Der Angreifer kann m durch Brute-Force über k und das Ziehen der e -ten Wurzel aus $c + kN$ rekonstruieren ($m = \sqrt[e]{c + kN}$).

Lösung: Padding

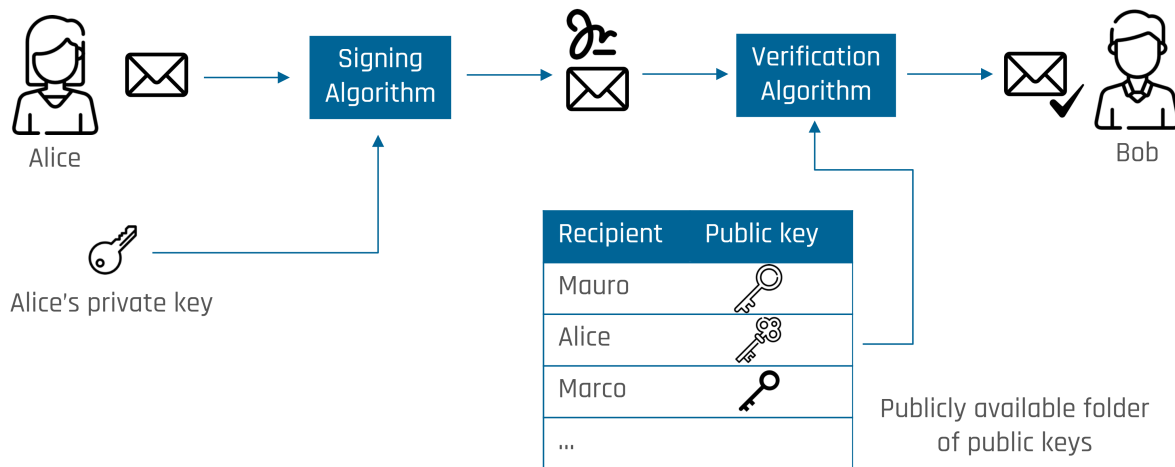
- Der Klartext m wird mit zufälligen Bytes aufgefüllt, um die Größe des Modulus N zu erreichen. Dies verhindert beide obigen Angriffe.
- **Algorithmen:** PKCS#1 v. 1.5 (hat bekannte Schwachstellen), **OAEP** (wird für neue Anwendungen empfohlen).

Digitale Signaturen

❓ Warum brauchen wir Digitale Signaturen?

Message Authentication Codes (MACs) bieten zwar Datenintegrität und Authentizität, können aber nur von Besitzern des geheimen Schlüssels verifiziert werden und bieten **keine** Non-Repudiation (Nicht-Abstreitbarkeit), da Sender und Empfänger den gleichen Key haben.

- Digitale Signaturen (asymmetrisch) bieten: Datenintegrität, Authentizität, öffentliche Verifizierbarkeit und **Non-Repudiation**.
- Der **Private Key** wird zum **Signieren** verwendet.
- Der **Public Key** wird zum **Überprüfen** (Verifizieren) verwendet.



Textbook RSA für Signaturen

Sei $m < N$ die zu signierende Nachricht und s die Signatur.

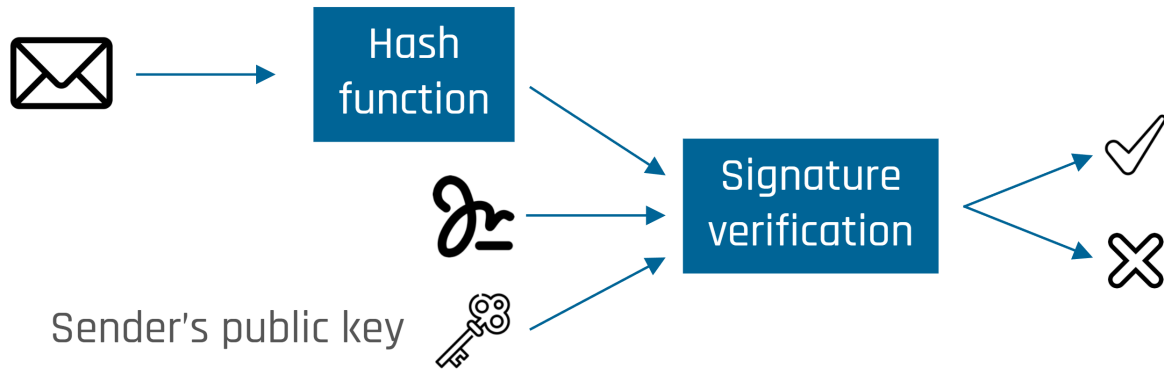
- **Signieren:** $s = m^d \bmod N$.
- **Verifizieren:** Überprüfe, ob $s^e \bmod N$ gleich m ist.

⚠ Schwachstellen von Textbook RSA Signaturen

- **Malleability (Verformbarkeit):** Aus gültigen Signaturen können neue gültige Signaturen für andere Nachrichten berechnet werden: $s_1 \cdot s_2 \bmod N = (m_1 \cdot m_2)^d \bmod N$ ist eine gültige Signatur für die Nachricht $m_1 \cdot m_2$.
- **Blinde Signaturen:** Ein Angreifer kann den Besitzer eines Private Keys austricksen, eine scheinbar harmlose Nachricht $m' = m \cdot r^e \bmod N$ signieren zu lassen, um daraus die Signatur für eine böartige Nachricht m zu extrahieren ($s = s' \cdot r^{-1} \bmod N$).
- **Längenbegrenzung:** Es können nur Nachrichten kleiner als N signiert werden. Splittet man die Nachricht in Blöcke und signiert diese einzeln, ist das extrem langsam, die Signatur wird riesig und man verliert die Integrität (Blöcke könnten einfach vertauscht werden, ähnlich wie beim ECB-Modus).

Lösung: Hash-and-Sign

Anstatt die Nachricht selbst zu signieren, wird der **Hash-Wert** der Nachricht signiert.



- **Vorteile:**

- Effizient für beliebig große Nachrichten (nur ein Hash-Vorgang und eine Signatur-Operation nötig).
- Die Signatur hängt von jedem Bit der Originalnachricht ab (Diffusion der Hashfunktion).
- Die **Kollisionsresistenz** der Hashfunktion verhindert, dass jemand eine andere Nachricht findet, für die dieselbe Signatur gilt.
- Verhindert Angriffe basierend auf Malleability (das Produkt zweier Hashes hat keine Beziehung zum Produkt der Nachrichten).
- **Padding:** Auch hier wird der Hash vor dem Signieren gepaddet (z.B. mit PSS, welches als sicher gilt). Hilft auch Manipulationen zu erkennen.

Diffie-Hellman Protokoll

Zweck

Ein Schlüsselaustausch-Protokoll, das es zwei Parteien erlaubt, über einen **öffentlichen (unsicheren) Kanal** ein **gemeinsames Geheimnis** (Shared Secret) zu generieren. Wurde 1976 veröffentlicht und basiert auf der Schwierigkeit, diskrete Logarithmen zu berechnen.

Ablauf

Beide Parteien einigen sich öffentlich auf zwei Parameter: eine große Primzahl p und eine Primitivwurzel g modulo p .

1. Alice:

- Wählt einen zufälligen privaten Wert $1 \leq prv_A < p$.
- Berechnet den öffentlichen Wert: $pub_A = g^{prv_A} \bmod p$ und sendet ihn an Bob.

2. Bob:

- Wählt einen zufälligen privaten Wert $1 \leq prv_B < p$.
- Berechnet seinen öffentlichen Wert: $pub_B = g^{prv_B} \bmod p$ und sendet ihn an Alice.

3. Gemeinsames Geheimnis S berechnen:

- Alice rechnet: $S = \text{pub}_B^{\text{prv}_A} \bmod p$.
- Bob rechnet: $S = \text{pub}_A^{\text{prv}_B} \bmod p$.
- Beide kommen auf den gleichen Wert S , der nun als symmetrischer Schlüssel genutzt werden kann.

Schwachstellen im Diffie-Hellman

• Schlechte Wahl der privaten Parameter:

- Wenn Alice $\text{prv}_A = 1$ wählt, ist $\text{pub}_A = g$. Das Geheimnis S wäre dann einfach pub_B , was öffentlich übertragen wurde (Angreifer kann mitlesen).
- Wenn Alice $\text{prv}_A = p - 1$ wählt, ist $\text{pub}_A = 1$. Das Geheimnis ist dann immer $S = 1$, egal was Bob wählt.
- **Lösung:** Der private Wert sollte immer aus der Menge $\{2, \dots, p - 2\}$ gewählt werden.

⚡ Man-in-the-Middle (MitM) Angriff

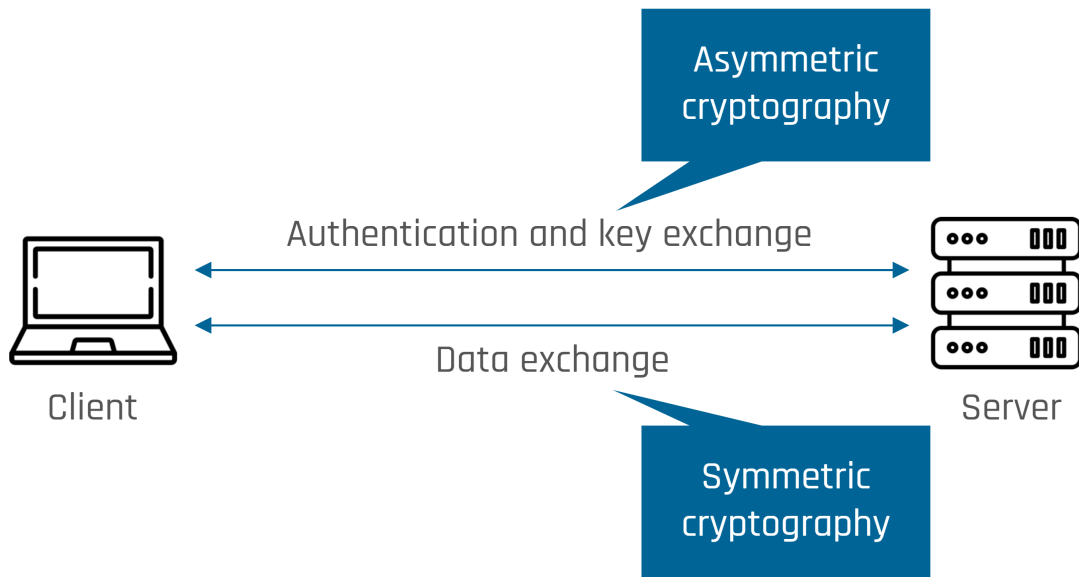
Da das Standard-Diffie-Hellman Protokoll **keine Authentifizierung** bietet, kann sich ein Angreifer dazwischenschalten.

- Der Angreifer fängt Alices pub_A ab und schickt ihr seinen eigenen pub_E .
- Er fängt Bobs pub_B ab und schickt ihm ebenfalls seinen eigenen pub_E .
- Ergebnis: Alice teilt ein Geheimnis mit dem Angreifer (S_{AE}) und Bob teilt ein Geheimnis mit dem Angreifer (S_{BE}). Der Angreifer kann nun alle Nachrichten mitlesen und weiterleiten.
- **Lösung:** Verwendung von **Zertifikaten** (Certificates) zur Authentifizierung.

Verbindungsaufbau im Internet (TL;DR)

In der Praxis (z.B. bei TLS/HTTPS) werden beide Welten kombiniert:

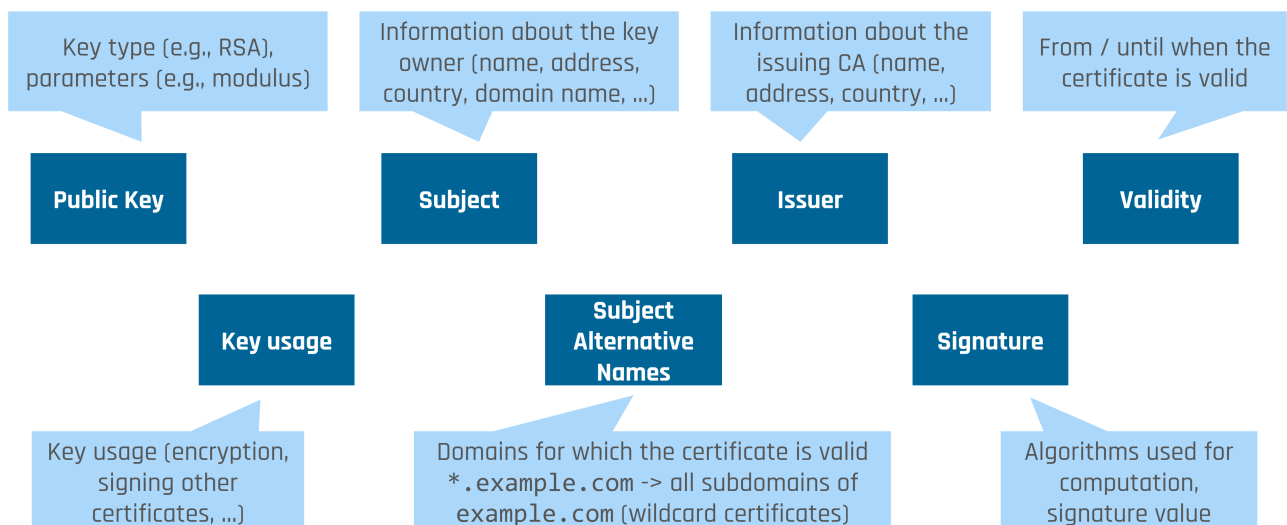
- **Asymmetrische Kryptographie:** Wird initial für die **Authentifizierung und den Schlüsselaustausch** (Key Exchange) verwendet.
- **Symmetrische Kryptographie:** Wird danach für den eigentlichen **Datenaustausch** (Data Exchange) genutzt, da sie viel schneller ist.



04_PKI_and_Crypto_Protocols

Public Key Certificates

- Ein grundlegendes Problem asymmetrischer Kryptographie ist die Zuordnung: Jeder könnte ein Schlüsselpaar erstellen und behaupten, der Public Key gehöre jemand anderem.
- Ein authentifizierter Schlüsselaustausch erfordert großen Aufwand (z. B. physisches Treffen), was über Distanz nicht praktikabel ist.
- **Public Key Certificates (Zertifikate)** binden einen Public Key kryptographisch an seinen Besitzer.
- Ein Zertifikat ist ein digitales Dokument zum Beweis der Authentizität eines Public Keys, typischerweise im Standard X.509 formatiert.
- Es wird in der Regel von einer vertrauenswürdigen Certificate Authority (CA) ausgestellt und mit deren privatem Schlüssel signiert.
- Wenn die Signatur gültig ist und man der CA vertraut, kann der Schlüssel sicher für die Kommunikation mit dem Zertifikatsinhaber (Subject) genutzt werden.



- **Inhalte eines Zertifikats:** Beinhaltet Public Key, Infos zum Besitzer (Subject), Infos zur ausstellenden CA (Issuer), Gültigkeitszeitraum, Key Usage (z. B. Verschlüsselung, Signieren), Subject Alternative Names (gültige Domains, z. B. *.example.com) und die Signatur der CA.
- **Erstellung:** Die Registration Authority (RA) prüft die Daten des Antragstellers, anschließend signiert die CA die Daten mit ihrem Private Key und fügt die Signatur ins Zertifikat ein.
- **Validierung:** Mit dem Public Key der CA wird geprüft, ob die Signatur zum Inhalt des Zertifikats passt; schlägt dies fehl, wurde das Zertifikat manipuliert oder gefälscht und wird als ungültig abgelehnt.

❓ **Woher wissen wir, dass der Public Key der CA authentisch ist?**

Die Authentizität wird durch eine **Certificate Chain** (Zertifikatskette) sichergestellt. Man validiert das Zertifikat der CA mit dem Public Key der übergeordneten CA, und so weiter, bis man bei einem Root-Zertifikat ankommt.

- **Self-signed Certificate**: Subject und Issuer sind dieselbe Entität (Signatur wird mit dem eigenen Public Key verifiziert); sie sind einfach zu erstellen, bieten aber keinen Trust-Wert.
- **Root Certificate**: Ein selbstsigniertes Zertifikat einer Root-CA, das als "Trust Anchor" des Systems dient.
- Betriebssysteme und Browser haben vertrauenswürdige Root-Zertifikate bereits vorinstalliert.

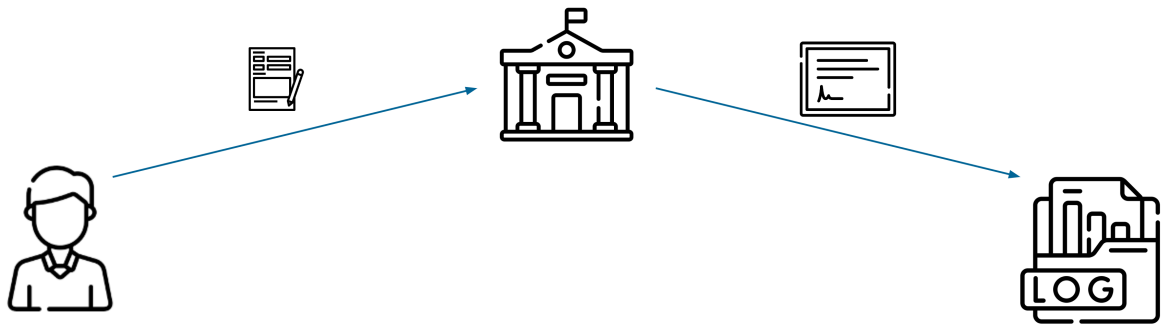
Public Key Infrastructures (PKI)

- Eine PKI ist ein Framework zur Ausstellung, Pflege, Verteilung und Revokation (Widerruf) von digitalen Zertifikaten.
- **Registration Authority (RA)**: Hier beantragen Personen/Entitäten Zertifikate. Die RA verifiziert die Korrektheit der Daten und leitet sie bei Erfolg an die CA weiter.
- **Verifizierungsstufen für TLS (RA-Aufgabe)**:
 1. **Domain Validated**: Nur die administrative Kontrolle über die Domain wird geprüft (z. B. via DNS TXT Record mit einem Token).
 2. **Organization-Validated**: Zusätzlich wird die Identität des Unternehmens hinter der Domain verifiziert und im Zertifikat vermerkt.
 3. **Extended Validation**: Noch detailliertere Hintergrundprüfungen des Unternehmens durch die RA.
- **Certificate Authority (CA)**: Stellt die Zertifikate für andere Subjekte aus (z. B. DigiCert, Let's Encrypt, Google).
- Die CA erstellt und managt auch eigene Schlüssel der PKI und kümmert sich um den Widerruf von Zertifikaten.
- **Certificate Revocation Lists (CRL)**: Listen von Zertifikaten, die vor Ablauf ungültig wurden (z. B. weil der Private Key geleakt ist oder sich Daten geändert haben).
- **Nachteile von CRLs**: Revokation ist unumkehrbar, Clients akzeptieren widerrufene Zertifikate bis zum nächsten CRL-Update, und die Listen werden extrem groß (Bandbreiten-intensiv).
- **Validation Services (z. B. OCSP)**: Alternative zu CRLs für Echtzeit-Abfragen des Revokationsstatus, bringt jedoch Latenzen beim Verbindungsaufbau und Datenschutzbedenken (CA erfährt, welche Website der Nutzer besucht) mit sich.
- **Certificate Policies**: Ein Dokument, das die Architektur, Prozesse (Registrierung, Schlüsselerstellung), Sicherheitsmechanismen und rechtlichen Rahmenbedingungen der PKI beschreibt, damit Außenstehende die Vertrauenswürdigkeit beurteilen können.

Certificate Transparency (CT)

- CAs können fälschlicherweise (durch Fehler oder Hacks) falsche Zertifikate ausstellen (Beispiele: DigiNotar 2011, Symantec 2015/2018).

- **Certificate Transparency (CT)** (2012) bekämpft dieses Problem: Alle von vertrauenswürdigen CAs ausgestellten Zertifikate müssen in kryptographisch geschützten, öffentlich einsehbaren "Append-only" Logs veröffentlicht werden.



- **Ablauf:**
 1. CA sendet ein "Pre-Certificate" (enthält eine "Poison Extension", damit es noch nicht nutzbar ist) an einen oder mehrere CT-Logs.
 2. Log verifiziert die Zertifikatskette und antwortet mit einem Signed Certificate Timestamp (SCT), was ein Versprechen ist, das Zertifikat innerhalb einer bestimmten Zeit (MMD) ins Log aufzunehmen.
 3. CA stellt das endgültige Zertifikat aus, welches die SCTs beinhaltet.
- CT-Monitore prüfen die Logs kontinuierlich, um Website-Betreiber über neue Zertifikate für ihre Domain zu informieren oder Konsistenzfehler der Logs aufzudecken.
- Browser (wie Chrome, Safari) erzwingen CT, indem sie Zertifikate ohne ausreichende Anzahl an SCTs (abhängig von der Laufzeit des Zertifikats) als unsicher markieren.

Cryptographic Protocols & Angriffe

⚠ Kryptographie allein reicht nicht aus!

Angreifer können Krypto-Systeme umgehen, indem sie Nachrichten auf Netzwerkebene abfangen, duplizieren oder zurücksenden, ohne die Verschlüsselung selbst knacken zu müssen.

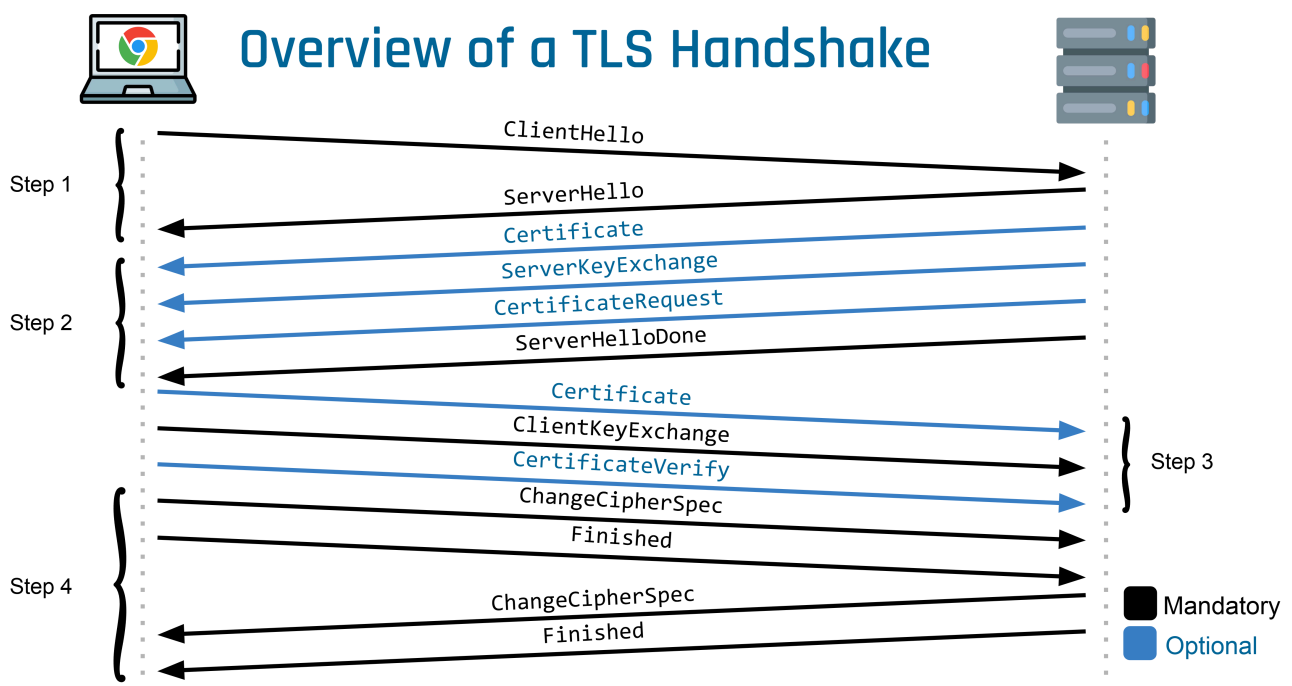
- **Interleaving Attack:** Eine Nachricht aus einer Session wird vom Angreifer genutzt, um mit einer anderen, parallel laufenden Session zu interagieren.
- **Reflection Attack:** Eine Form der Interleaving Attack, bei der eine Nachricht an ihren Erzeuger zurückgeschickt wird.
- Reflection Attacks funktionieren besonders bei symmetrischer Verschlüsselung, wenn das Opfer mehrere Rollen im Protokoll einnimmt (z.B. Alice kann nicht unterscheiden, ob ein verschlüsselter Befehl von Bob oder von ihr selbst reflektiert wurde).
- **Lösung gegen Reflection:** Brechen der Symmetrie, indem man den Identifikator des Senders (oder Empfängers) mit in die verschlüsselte Nachricht packt (z. B. $\$ \{Alice, "Give\ Eve\ 1000\$"\}_k \$$).

- **Replay Attack:** Eine exakte Kopie einer gültigen Nachricht wird abgefangen und mehrfach an den Empfänger gesendet (z. B. eine Geldüberweisung mehrfach auslösen).
- **Lösung gegen Replay:** Nutzung eines Challenge-Response-Verfahrens mit einer **Nonce** (Number used once). Die Nonce n ist eine Zufallszahl, die nur für eine Session gültig ist; eine Anfrage wird nur akzeptiert, wenn die Nonce noch nie zuvor verwendet wurde.

Transport Layer Security (TLS)

- TLS ist das Standardprotokoll für sichere Datenübertragung im Internet (früher SSL).
- Am weitesten verbreitet ist TLS 1.2, die aktuellste Version ist TLS 1.3.
- **Sicherheitsgarantien:**
 - **Confidentiality** (Vertraulichkeit) durch symmetrische Verschlüsselung.
 - **Integrity** (Integrität) der Daten durch MACs.
 - **Authentication** (Authentifizierung) der Partner mittels asymmetrischer Algorithmen und Zertifikaten (Server-Auth ist fast immer aktiv, Client-Auth optional).

TLS Handshake (Version 1.2)



- Der Verbindungsaufbau bei TLS 1.2 erfolgt in 4 Schritten.

1. Negotiation of Security Parameters (Aushandlung):

- Client sendet **ClientHello**: Beinhaltet die höchste unterstützte TLS-Version, unterstützte Cipher Suites, Kompressionsmethoden und eine Zufallszahl (Client Nonce).
- Server antwortet mit **ServerHello**: Beinhaltet die final gewählte TLS-Version, gewählte Cipher Suite und eine eigene Zufallszahl (Server Nonce).

2. Server Authentication and Key Exchange:

- Server sendet **Certificate**: Beinhaltet die Zertifikatskette des Servers.

- Server sendet `ServerKeyExchange` (optional): Wird gesendet, wenn z.B. Ephemeral Diffie-Hellman genutzt wird; enthält Modulus P , Generator g , den DH Public Value des Servers $g^a \bmod P$ und eine Signatur des Servers über diese Werte plus die Nonces aus Schritt 1.
- Server sendet `ServerHelloDone`: Signalisiert das Ende von Schritt 2. Der Client verifiziert nun das Zertifikat.

3. Client Key Exchange (und optionale Client Auth):

- Client sendet `ClientKeyExchange`: Enthält bei Ephemeral Diffie-Hellman den DH Public Value des Clients $g^b \bmod P$, wobei $b \in [2; P - 2]$ vom Client gewählt wurde.
- Nun können Client und Server beide aus den DH-Werten das **Premaster Secret** berechnen (g^{ab}).
- Aus dem Premaster Secret und den beiden Nonces (aus Schritt 1) wird das eigentliche **Master Secret** abgeleitet, aus dem alle weiteren Schlüssel (Verschlüsselung, HMAC) generiert werden.

4. Completion of the Handshake:

- Beide Seiten senden nacheinander `ChangeCipherSpec` (ab jetzt werden die ausgehandelten Krypto-Algorithmen verwendet) und `Finished`.
- Die `Finished`-Nachricht ist die erste verschlüsselte Nachricht und enthält einen HMAC über alle bisherigen Handshake-Nachrichten, um sicherzustellen, dass niemand (z. B. ein Man-in-the-Middle) die Parameter im Klartext verändert hat.

Was ist eine Cipher Suite?

Ein String, der 4 Krypto-Mechanismen definiert. Beispiel:

`TLS_DHE_RSA_WITH_AES_256_CBC_SHA384`

- `DHE`: Key Exchange Algorithm (Ephemeral Diffie-Hellman).
- `RSA`: Authentication Mechanism (Zertifikate).
- `AES_256_CBC`: Symmetric Encryption Algorithm & Mode.
- `SHA384`: Hash-Funktion (für HMACs und Key Derivation).

Forward Secrecy

Forward Secrecy bedeutet, dass vergangene aufgezeichnete Verbindungen nicht nachträglich entschlüsselt werden können, selbst wenn der private RSA-Schlüssel des Servers in der Zukunft geleakt wird.

Erreicht wird dies durch *Ephemeral* (flüchtiges) Diffie-Hellman (`DHE` oder `ECDHE`). Für jede Session werden neue DH-Schlüssel (a und b) generiert und am Ende verworfen. Der langlebige RSA-Schlüssel wird nur für die Signatur/Authentifizierung beim Handshake verwendet, nicht für die Verschlüsselung!

05_Authentication

Grundlagen der Sicherheit: Identification vs. Authentication

Wichtige Unterscheidung

Identifikation und **Authentifizierung** sind zwei grundlegend verschiedene, aber aufeinander aufbauende Schritte im Zugriffsprozess.

- **Identifikation (Identification):**

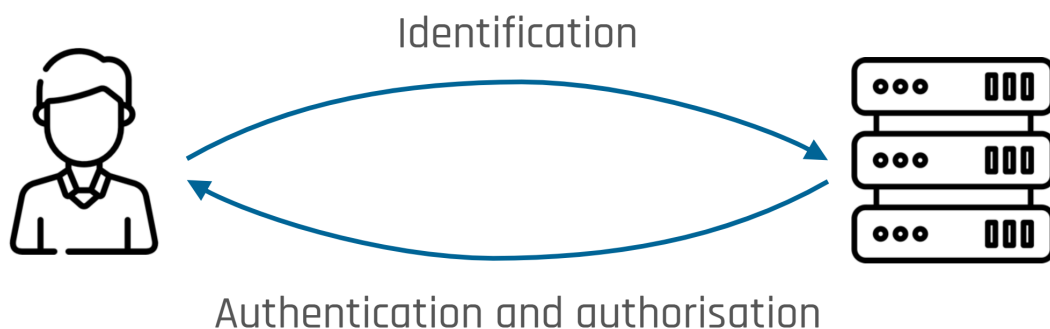
- Prozess, bei dem die Identität einer Person durch unterscheidende Informationen angegeben wird.
- *Physische Welt:* Name, Geburtsdatum/-ort (Eindeutigkeit durch Standesämter).
- *Digitale Welt:* E-Mail-Adresse, selbstgewählter oder automatisch zugewiesener Benutzername.

- **Authentifizierung (Authentication):**

- Der Akt der Überprüfung / Verifizierung dieser behaupteten Identität.
- *Physische Welt:* Vorzeigen eines Personalausweises (Vergleich der Infos).
- *Digitale Welt:* Passwörter, Biometrie, Zertifikatsüberprüfung.

Steps to Access an IT System

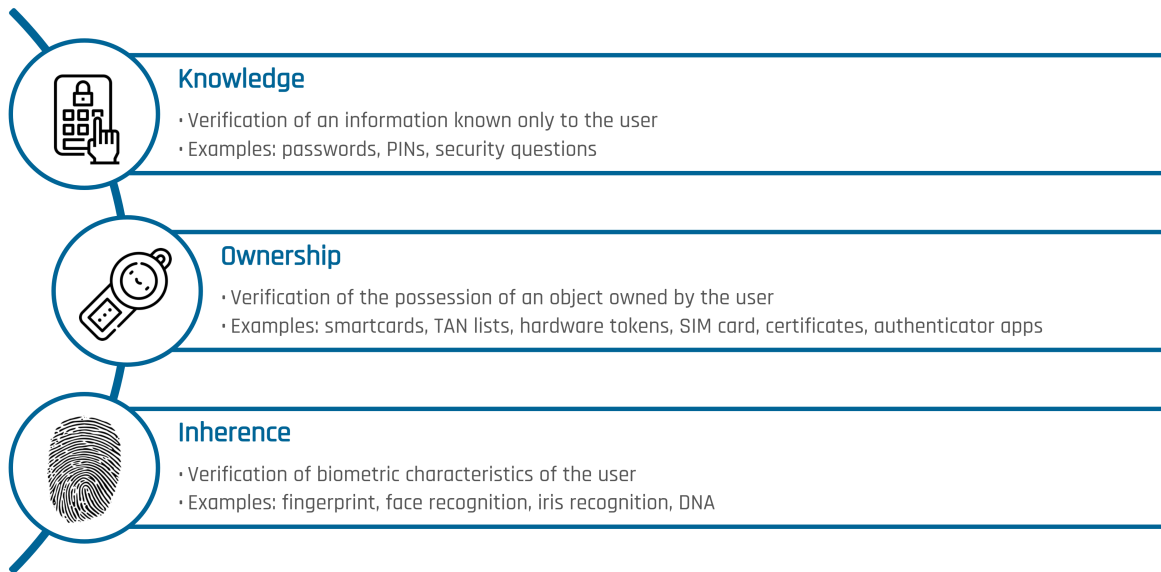
1. **Identifikation:** Der Nutzer liefert Identitätsinformationen und einen Echtheitsnachweis.
2. **Authentifizierung:** Das System verifiziert die Korrektheit der gelieferten Daten.
3. **Autorisierung (Authorisation):** Der Zugriff auf das System wird *nach* erfolgreicher Authentifizierung gewährt.



Klassen von Authentifizierungsmethoden

Authentifizierungsfaktoren lassen sich in drei Hauptkategorien einteilen:

1. **Wissen (Knowledge):** *Überprüfung einer Information, die nur* der Nutzer kennt.*
 - *Beispiele:* Passwörter, PINs, Sicherheitsfragen.
2. **Besitz (Ownership):** * Überprüfung des Besitzes eines Objekts.
 - *Beispiele:* Smartcards, TAN-Listen, Hardware-Token, SIM-Karten, Zertifikate, Authenticator Apps.
3. **Inhärenz (Inherence):** * Überprüfung biometrischer Eigenschaften des Nutzers (körperlich oder verhaltensbedingt).
 - *Beispiele:* Fingerabdruck, Gesichtserkennung, Iris-Erkennung, DNA.



Passwort-basierte Authentifizierung

Zusammenfassung

Die am weitesten verbreitete Methode im Internet. Der Nutzer registriert sich vorab mit Name/Passwort, welches beim Login abgefragt wird. Ein solches System erfordert die Verwaltung einer Passwortliste.

- **Sicherheitsfaktoren:** Die Sicherheit hängt kritisch von der Geheimhaltung und Qualität des Passworts sowie dessen Speicherung und Übertragung ab.
- **Sichere Übertragung:** Passwörter müssen zwingend sicher kommuniziert werden (z.B. über TLS), um Leaks durch Netzwerk-Angreifer zu verhindern.

Angriffsvektoren (Attack Vectors)

- **Raten (Guessing):** Trial-and-Error oder Social Engineering.

- **Abfangen (Interception):** Während der Eingabe (z.B. Keylogger, Shoulder Surfing) oder Übertragung (Netzwerkangriff).
- **Diebstahl (Theft):** Entwenden der Passwortliste direkt vom System.

Arten von Angriffen auf Passwörter

- **Offline-Angriffe:** Angreifer hat Zugriff auf die gespeicherte (verschlüsselte/gehashte) Passwortliste und versucht, diese in Ruhe wiederherzustellen.
- **Online-Angriffe:** Angreifer interagiert direkt mit dem Ziel-Service (z.B. Login-Formular).

Strategien zum Knacken

1. Wörterbuch-Angriffe (Dictionary-based):

- Ausprobieren von häufig genutzten Passwörtern aus einer vorbereiteten Liste (Dictionary), meist aus früheren Leaks.
- Wird oft mit Social Engineering kombiniert (Namen von Kindern, Haustieren, etc.).
- **Variationen:** Zahlen/Satzzeichen werden ergänzt, oder Buchstaben durch ähnlich aussehende Zahlen ersetzt (z.B. $s \rightarrow 5$, $E \rightarrow 3$).

2. Brute-Force-Angriffe:

- Systematisches Testen **aller** möglichen Kombinationen.
- Theoretisch immer erfolgreich (bei unendlicher Zeit), praktisch aber nur für Offline-Angriffe realistisch.

Alphabet	Password length	# combinations	Full search time
0-9 (10 chars)	8	$10^8 = 100.000.000$	100 seconds
	10	$10^{10} = 10.000.000.000$	2,8 hours
A-Z a-z 0-9 (62 chars)	8	$62^8 = 56.800.235.584$	6,9 years
	10	$62^{10} = 839.299.365.868.340.244$	26.614 years
A-Z a-z 0-9 [][]{} ?!\$%&/=*+~.,,:;<> -_ (86 chars)	8	$86^8 = 2.992.179.271.065.856$	94,88 years
	10	$86^{10} = 22.130.157.888.803.070.976$	701.743 years

Speicherung von Passwörtern (Password Storage)

⚡ How NOT to

Klartext: Wenn die Liste leakt, sind alle Konten kompromittiert! (z.B. Facebook 2019, A1 2018).

Verschlüsselt: Der Entschlüsselungs-Key muss auf dem System liegen. Leakt dieser Key

zusammen mit der Liste, können ebenfalls alle Passwörter wiederhergestellt werden (z.B. Adobe Leak 2013).

Hashing

- **Funktionsweise:** Es wird nur der **Hash** (Prüfsumme) des Passworts gespeichert, berechnet durch eine kryptographische Hash-Funktion (Einwegfunktion, nicht umkehrbar).
- Beim Login wird der Hash der Eingabe mit dem gespeicherten Hash verglichen.
- **Angriff:** Angreifer berechnen Hashes von häufigen Passwörtern und vergleichen diese mit der Liste (Offline-Wörterbuch-Angriff).
- **Rainbow Tables:** Vorausberechnete Tabellen, die Hash-Werte auf Passwörter mappen.

Salt and Pepper

Um Rainbow Tables und vorberechnete Angriffe unbrauchbar zu machen:

- **Salt:** Ein zufällig generierter String, der *für jeden Nutzer unterschiedlich* ist und mit dem Passwort vor dem Hashing verkettet wird.
 - Verhindert die Nutzung von Rainbow Tables (da Angreifer Hashes für jeden möglichen Salt vorberechnen müssten).
 - Verbirgt Passwort-Wiederverwendung (gleiches Passwort + unterschiedlicher Salt = unterschiedlicher Hash).
 - *Formel:* $Hash = h(Passwort || Salt || Pepper)$.
- **Pepper:** Ein *geheimer*, für alle Nutzer gleicher String, der außerhalb der Passwortliste (z.B. in einem HSM) gespeichert wird.
 - Macht das Knacken selbst eines einzelnen Hashes (und bei schwachen Passwörtern) unmöglich, solange der Pepper geheim bleibt.
 - NIST empfiehlt eine Mindestgröße von 112 Bits.

Cascade Hashing & KDFs

- **Cascade Hashing:** Mehrfache, wiederholte Anwendung der Hash-Funktion (z.B. 100k bis 1 Million mal), um die Berechnung künstlich zu verlangsamen.
 - Führt zu einer winzigen Verzögerung für den echten Nutzer (<1s), erhöht aber den Aufwand für Brute-Force-Angreifer extrem (Faktor c).
- **PBKDF2 (Password-based Key Derivation Function):**
 - Leitet kryptographischen Schlüssel aus Passwort, Salt, und Iterationszahl c mithilfe einer PRF (meist HMAC) ab.
 - *Schwachstelle:* Benötigt wenig Arbeitsspeicher und ist daher leicht parallelisierbar (Angriffe über GPUs/ASICs).
- **Moderne KDFs (scrypt, Argon2, yescrypt):**
 - Erlauben auch die Konfiguration der **Speichernutzung** (Memory-Hardness).

- Verhindern hochgradig parallelisierte Angriffe, da diese entweder enormen RAM oder extrem lange Rechenzeiten fordern würden.

Speicherung in UNIX/Linux

- Zugangsdaten liegen in `/etc/shadow`.
 - **Format:** `$id$rounds=xxx$salt$hashedpwd$`.
 - `id` : verwendeter Algorithmus (1=MD5, 5=SHA-256, 6=SHA-512, y=yescrypt).
 - `rounds` : (optional) Iterationsanzahl (Standard oft 5000).
 - `salt` : Nutzer-individueller Salt.
-

Best Practices & Abwehrmaßnahmen

Schutz vor Online-Angriffen

- Erzwingen von Verzögerungen (Delays) nach fehlerhaften Versuchen (wachsend).
- Account-Sperre nach Limit (Achtung: Kann für Denial-of-Service (DOS) missbraucht werden!).
- CAPTCHAs (Beweis für menschlichen Nutzer).
- **Multi-Faktor-Authentifizierung (MFA)**.

Passwort-Richtlinien für User

- **Länge schlägt Komplexität:** NIST empfiehlt mind. 8 Zeichen, besser länger. ("correct horse battery staple" ist besser merkbar und schwerer zu knacken als "Troub4dor&3").
- Alphabet: Ziffern, Groß-/Kleinbuchstaben (gegen Brute-Force).
- Keine einzelnen, gewöhnlichen Wörter verwenden (gegen Wörterbuch-Angriffe).
- **Niemals Passwörter wiederverwenden!** (Leak bei Service A kompromittiert Service B).
- Nutzung von **Passwort-Managern** (z.B. KeePass (Desktop/Offline), Bitwarden/1Password (Cloud/Sync)).

Überholte und schlechte Praktiken

- **Passwort-Rotation (Verpflichtender Wechsel nach X Tagen):** NIST rät mittlerweile *davon ab*, da Nutzer dazu tendieren, schwächere Passwörter zu wählen oder diese aufzuschreiben (z.B. Post-Its).
- **Sicherheitsfragen (Password Recovery Questions):** Sind extrem unsicher, da Antworten leicht über Social Engineering oder Social Media (Facebook/Instagram) herausgefunden werden können (z.B. Mädchenname der Mutter). Falls möglich: Deaktivieren!

Password Theft (Diebstahlarten)

- **Shoulder Surfing:** Beobachten der Eingabe im öffentlichen Raum (Blick über die Schulter).
- **Keyloggers:** Malware, die Tastenanschläge aufzeichnet und an Angreifer ("Drop Zones") sendet.
- **Phishing:** Angreifer gibt sich als vertrauenswürdige Entität aus (E-Mail, Telefon), um das Opfer auf gefälschte Websites zu locken.
 - *Spear Phishing:* Gezielte, auf das spezifische Opfer maßgeschneiderte Angriffe.

(Info: Mit Tools wie "Have I Been Pwned" kann man prüfen, ob die eigene E-Mail in einem Breach geleakt wurde. Hashes werden lokal im Browser berechnet und sicher abgefragt).

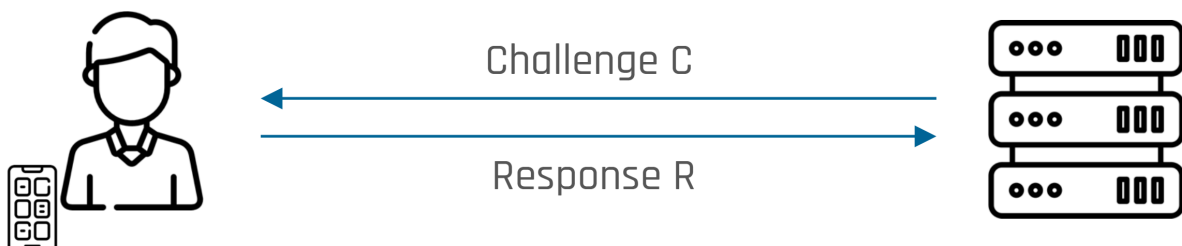
OTP, Challenge-Response & MFA

One-Time Passwords (OTP)

- Passwörter, die **nur ein einziges Mal** verwendet werden können (schützt vor Replay-Attacks).
- *TAN-Listen:* Vorausberechnet und sicher übermittelt (z.B. per Post). Die Anzahl der Transaktionen ist durch die Liste limitiert.
- *OTP Generators:* Das Passwort wird live beim Login kryptographisch berechnet.
 - Oft zeitbasiert und abhängig von einem geheimen User-Schlüssel S_X (z.B. $OTP = h(Time || S_X)$).
 - Umgesetzt per Hardware-Token oder Authenticator App.
- *Mobile TAN (SMS):* Code wird per SMS gesendet. Beweist den Besitz der SIM-Karte.

Challenge-Response Methods

- Ähnlich zu OTP-Generatoren, jedoch sendet der Server hier eine dynamische **Challenge C** (mit Zufallskomponente gegen Replay-Angriffe) an den Nutzer.
- Nutzer / App berechnet die **Response R** durch kryptographische Operationen.
- *Implementierungsarten:*
 - Hash-basiert: $R = h(C || S_X)$
 - Symmetrische Verschlüsselung (z.B. AES): $R = Enc(K_X, C)$
 - Digitale Signatur (z.B. RSA): $R = Sign(SK_X, C)$



Hardware-Token

- **Disconnected Tokens:** Zeigen Code auf einem kleinen Display an (reine OTP-Generatoren).

- **Connected Tokens:** Werden physisch ans Gerät angeschlossen (USB, NFC, Bluetooth). Genutzt für OTP oder Challenge-Response (WebAuthn).
- **Vorteil:** Kein Merken von Passwörtern nötig, schützt bei Passwort-Reuse in MFA.
- **Nachteil:** Bei Verlust/Diebstahl kompletter Sicherheitsverlust (falls als einziger Faktor genutzt).

Biometrie (Biometrics)

Nutzt biologische oder verhaltensbedingte Eigenschaften zur Authentifizierung.

- **Passiv:** Fingerabdruck, Gesicht, Iris, Retina, Handgeometrie, Venen.
- **Aktiv:** Stimmerkennung, Tastenanschlagdynamik (Keystroke dynamics), Mausbewegung, Gangart (Gait).

Gewünschte Eigenschaften biometrischer Merkmale

- **Universality (Universalität):** Jeder Nutzer besitzt das Merkmal.
- **Uniqueness (Einzigartigkeit):** Unterscheidbarkeit zwischen Individuen.
- **Measurability (Messbarkeit):** Kann quantitativ erfasst werden.
- **Permanence (Beständigkeit):** Verändert sich nicht stark über die Zeit.
- **Unforgeability (Fälschungssicherheit):** Schwer zu imitieren.
- **Performance:** Schnell, effizient und genau auszuwerten.

Fehlerraten & Evaluation

Toleranzen bei der Messung (Brille, Schmutz etc.) führen zu Fehlern:

- **FAR (False Acceptance Rate) - misst die Sicherheit:** Wie viele *unautorisierte* Nutzer werden fälschlicherweise zugelassen?
- **FRR (False Rejection Rate) - misst die Usability:** Wie viele *autorisierte* Nutzer werden fälschlicherweise abgelehnt?
- **EER (Equal Error Rate):** Der Schnittpunkt, an dem $FAR = FRR$. Stellt einen guten Kompromiss zwischen Sicherheit und Komfort dar.

Biometric feature	Typical FAR (in %)	Typical FRR (in %)
Fingerprint	0,001 ... 2	0,1 ... 5
Iris recognition	0,0001 ... 1	0,1 ... 2
Face recognition	0,5 ... 2	1 ... 3
Hand geometry	1 ... 4	1 ... 5

Biometrische Merkmale können nachgemacht werden (Masken, Fotos, Silikonfinger).

Gegenmaßnahme: **Liveness Detection** (Prüfung auf Blutfluss, Pupillenbewegung).

Ein massives Problem ist zudem, dass Biometrie **nicht widerrufbar** (non-revocable) ist. Wenn der Fingerabdruck einmal geleakt ist, kann man ihn nicht einfach ändern. Aus Privacy-Gründen daher für Internet-Auth oft ungeeignet.

Multi-Factor Authentication (MFA)

Nutzung von *mehreren Authentifizierungsmethoden aus UNTERSCHIEDLICHEN Klassen* (Wissen, Besitz, Inhärenz).

- **Ziel:** Die Schwächen der einen Methode durch die Stärken der anderen auszugleichen.
- **Beispiel:** Passwort (Wissen) + Smartphone Authenticator (Besitz).
- **WICHTIG:** Passwort + PIN sind *kein* MFA, da beides aus der Klasse "Wissen" stammt.

Risk-Based Authentication

Die geforderten Authentifizierungsschritte richten sich nach dem Risiko der aktuellen Aktion (abhängig von Standort, Vertrauenswürdigkeit, Gewohnheiten, Kritikalität der Operation).

(Bsp: Login aus Heimatstadt -> nur Passwort. Überweisung in ein Drittland -> Passwort + Hardware-Token + Bestätigungsanruf).

Single Sign-On (SSO)

Nutzer authentifizieren sich bei vielen **Service Providern (SP)**, indem sie den Authentifizierungsprozess an einen zentralen **Identity Provider (IDP)** (z.B. Google, Facebook, GitHub) delegieren.

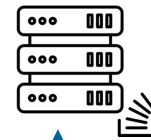
- Protokolle: OpenID Connect (basierend auf OAuth 2.0), SAML.

Ablauf (OpenID Connect - Authorisation Code Flow)

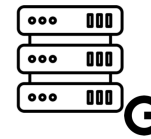
1. Nutzer will sich bei SP (z.B. StackOverflow) anmelden und wird zum IDP (Google) weitergeleitet.
2. Nutzer authentifiziert sich direkt beim IDP.
3. IDP leitet Nutzer zurück zum SP, inklusive eines **Authorisation Codes** in der URL.
4. SP sendet diesen Code im Hintergrund an den IDP.
5. IDP validiert den Code und stellt ein **Identity Token** für den SP aus, welches die Nutzerinformationen enthält.

```
{  
  "iss": "https://idp.google.com",  
  "sub": "g34nth543",  
  "aud": "nbj593feD",  
  "exp": 1311281970,  
  "iat": 1311280970,  
  "email": "aschmitt@gmail.com",  
  "given_name": "Andreas",  
  "family_name": "Schmitt"  
}
```

Identity Provider **validates** the received code and issues an **identity token** for the user



Service Provider
(StackOverflow)



Identity Provider
(Google)

✓ Vorteile SSO

Nutzer müssen sich nur noch ein Credential merken und SPs müssen keine eigene Login-Infrastruktur verwalten.

⚡ Nachteile SSO

Der IDP wird zum **Single Point of Failure** (Fällt er aus, gehen keine Logins mehr). Werden die Zugangsdaten zum IDP gestohlen, sind **alle** damit verknüpften Service Provider kompromittiert.

06_Access_Control_and_Containers

Access Control (Zugriffskontrolle)

Access Control

Die Entscheidung, ob einem Subjekt der Zugriff auf Systemobjekte (Netzwerk, Daten, Anwendungen, etc.) erlaubt oder verweigert wird.

- **Subjekte:** Entitäten, die auf Ressourcen zugreifen wollen (z. B. Benutzer, Prozesse).
- **Objekte:** Ressourcen, die geschützt werden müssen (z. B. Dateien, Peripheriegeräte).
- **Zugriffsrechte (Access rights):** Art des erlaubten Zugriffs (z. B. Lesen, Schreiben, Ausführen, Erstellen, Löschen).

Durchsetzung der Zugriffskontrolle (Enforcement)

Reference Monitor

Die Systemkomponente, die Zugriffsanfragen autorisiert oder blockiert (z. B. das Dateisystem im Betriebssystem).

Ein Reference Monitor benötigt **drei zwingende Eigenschaften**:

1. **Non-bypassable (Nicht umgehbar):** Alle Anfragen *müssen* durch den Monitor. Verhindert unerwartete Richtlinienviolationen.
2. **Verifiable (Überprüfbar):** Der Monitor muss analysierbar und testbar sein, um korrekte Durchsetzung zu garantieren.
3. **Tamper-proof (Manipulationssicher):** Darf von einem Angreifer nicht kompromittiert werden können.

Security Policy (Sicherheitsrichtlinie)

- Regelwerk, das festlegt, *welche* Rechte *welche* Subjekte auf *welchen* Objekten haben.
- **Delegation:** Darf ein Subjekt seine Rechte an andere weitergeben?
- **Revocation (Widerruf):** Wie können einem Subjekt Rechte wieder entzogen werden?

Modelle zur Speicherung von Zugriffsrechten

Access Control Matrix

- Entwickelt von Lampson (1971).
- **Zeilen** = Subjekte, **Spalten** = Objekte.
- Der **Schnittpunkt** (Zelle) definiert die Rechte.

Objects

*(User 1, File 1) = "read":
User 1 can read File 1*

	File 1	File 2	File 3	...	File n
User 1	read	write	-	-	read
User 2	write	write	write	-	-
User 3	-	-	-	read	read
...					
User m	read	write	read	write	read

Subjects

ACLs und Capabilities

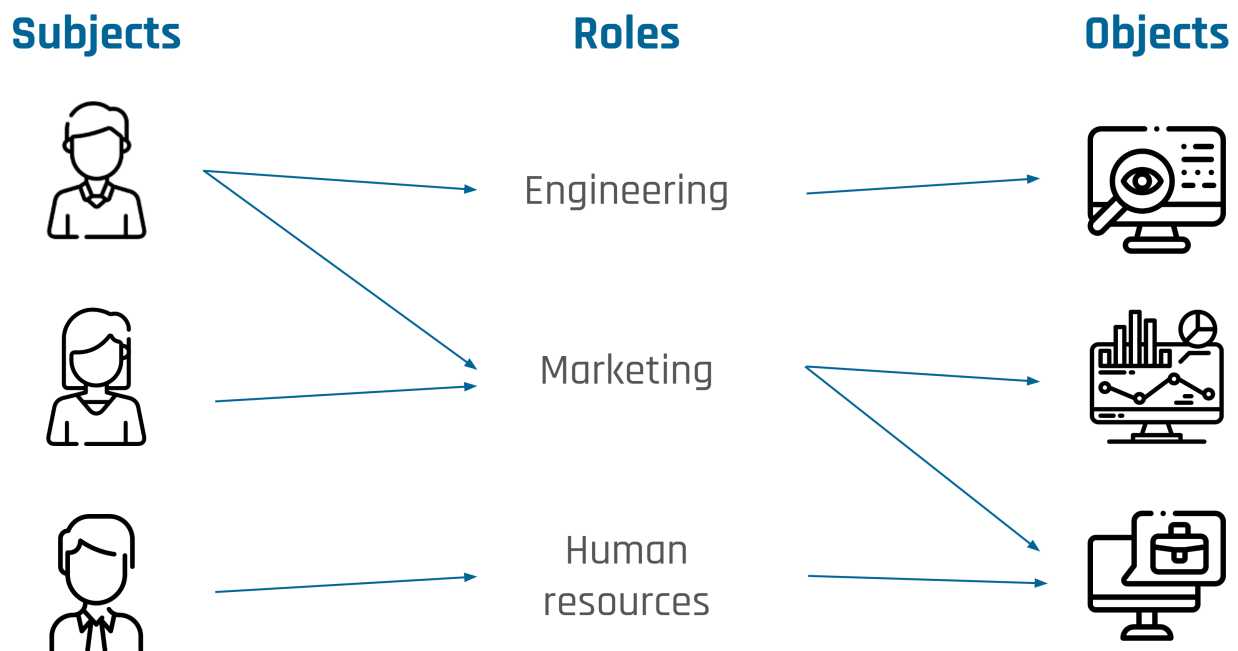
Vergleich von Speichermodellen

Wie werden Rechte technisch am Objekt oder Subjekt geprüft?

- **Access Control Lists (ACL):**
 - **Objekt-zentriert:** Jedes Objekt hat eine Liste berechtigter Subjekte.
 - Monitor prüft das anfragende Subjekt gegen die Liste des Objekts.
 - *Nachteil:* Erfordert Authentifizierung (das System muss den Nutzer kennen).
 - *Delegation:* Administrator fragen oder über OS-Mechanismen Prozesse unter anderer Nutzer-ID ausführen.
 - *Revocation:* ACL des Objekts wird einfach modifiziert.
- **Capabilities:**
 - **Subjekt-zentriert:** Nutzer erhält ein unfälschbares Ticket/Token, das Privilegien definiert.
 - Implementierung oft über kryptografische Zufalls-Strings (HMAC) oder direkt durchs OS.
 - Monitor prüft *nur* die Gültigkeit des Tokens (Identität des Subjekts ist egal).
 - *Delegation:* Token wird zur Laufzeit einfach an andere weitergereicht.
 - *Revocation:* Schwerer. Monitor muss widerrufene Tokens bis zum Ablaufdatum "tracken" (Bookkeeping). Wird ein Token für *mehrere* Ressourcen genutzt, werden bei Widerruf immer *alle* Rechte entzogen.

Arten der Zugriffskontrolle (MAC vs. DAC vs. RBAC)

- **Mandatory Access Control (MAC):**
 - Zentral gesteuert durch vertrauenswürdigen Administrator.
 - Subjekte können die Richtlinien *nicht* umgehen/überschreiben.
- **Discretionary Access Control (DAC):**
 - Nutzer können Rechte für Objekte, die sie besitzen (oder für die sie Capabilities haben), **frei delegieren, ändern oder widerrufen**.
- **Role-Based Access Control (RBAC):**
 - Löst Problem der Rechteverwaltung in großen Systemen.
 - **Rollen (Gruppen)** fassen Subjekte mit ähnlichen Rechten zusammen.
 - Berechtigungen → Rolle → Subjekte.
 - *Hierarchien*: Partielle Ordnung. Eine höhere Rolle (z. B. PowerUser) erbt alle Rechte der darunterliegenden Rollen (z. B. Guest) und bekommt nur *neue* Rechte spezifisch zugewiesen.

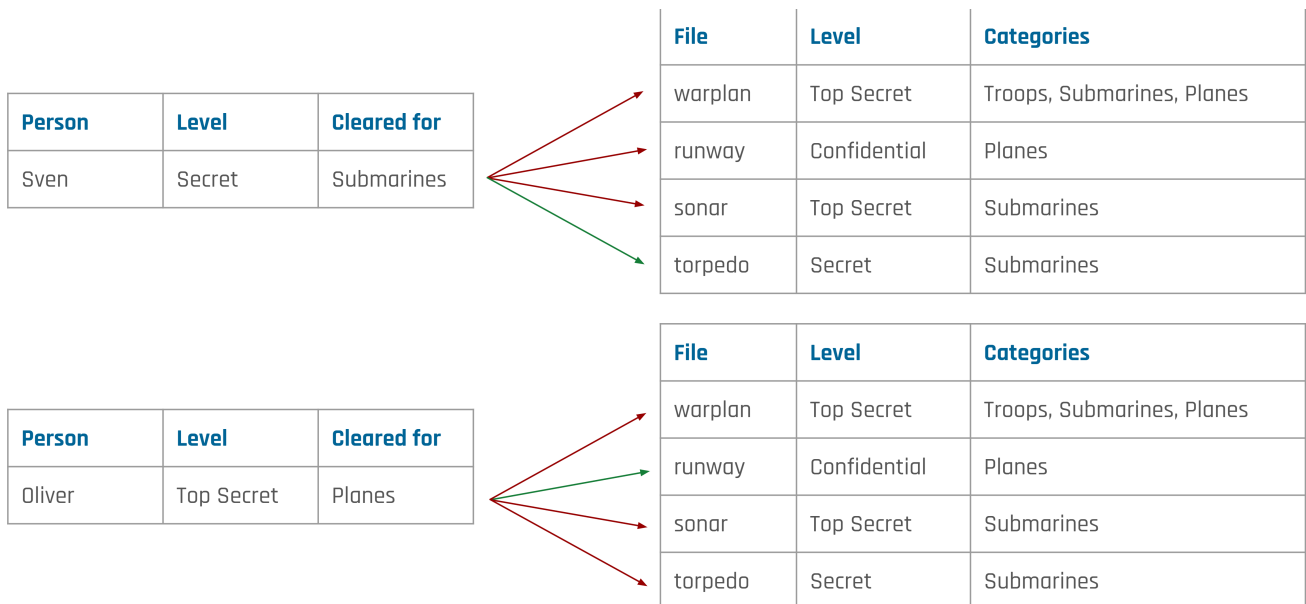


Spezifische Security Policy Modelle

Bell-LaPadula Modell (Vertraulichkeit)

- Ursprung: Militär / Regierung.
- **Fokus:** Daten-Vertraulichkeit (Confidentiality) wahren.
- Subjekte/Objekte haben **Security Level** (z. B. Secret, Top Secret) und **Kategorien** (z. B. Submarines).
- **Regeln:**

- **Lesen ("No read up"):** Subjekt Level $\geq$ Objekt Level **UND** Subjekt muss für alle Kategorien des Objekts freigegeben sein ($\supseteq$).
- **Schreiben ("No write down"):** Subjekt Level $\leq$ Objekt Level **UND** Subjekt-Kategorien $\subseteq$ Objekt-Kategorien. Verhindert das unbefugte "Herabstufen" von geheimen Informationen.



Weitere Policies

- **Biba Modell (Integrität):** Gegenteil von Bell-LaPadula. *"No read down, no write up"*. Schützt Daten vor Manipulation durch niedrigere Stufen.
- **Conditional Policies (Bedingt):**
 - Temporal: Erlaubt z. B. nur Zugriff zwischen 10:00 und 18:00 Uhr.
 - Context-aware: Nur aus dem Büro-Netzwerk erlaubt.
- **Separation of Duty (Vier-Augen-Prinzip):** Kritische Tasks (z. B. Scheck > \$10.000) brauchen Aktionen von zwei *verschiedenen* berechtigten Personen.
- **Chinese Wall Policy:** Verhindert Informationsfluss, der Interessenkonflikte auslöst (Bsp: Ein Anwalt darf nicht beide verfeindeten Parteien in einem Fall vertreten).

UNIX File System Access Control

- Jede Datei gehört einem **Owner (Besitzer)** und einer **Group (Gruppe)**.
- Rechte-Darstellung durch **3 Triaden**: **Owner** | **Group** | **Others**.
- **Rechte:** **r** (read), **w** (write), **x** (execute).
- **Auflösungsreihenfolge:** Bist du Owner? → nutze Owner-Rechte. Sonst: In der Gruppe? → nutze Gruppen-Rechte. Sonst: nutze Others-Rechte. (*Tip: Owner-Rechte gelten, auch wenn sie strikter als die Gruppenrechte sind!*)

 **Superuser (root)**

UID = 0. Hat volle Rechte auf dem System und ignoriert sämtliche Datei-Berechtigungen.
Normalerweise Login blockiert; Zugriff über `sudo` (geregelt in `/etc/sudoers`).

Notation der Berechtigungen

- **Symbolisch:** `rwxr-x---`.
- **Oktal:** z. B. `750` (`4` =read, `2` =write, `1` =execute). Addiert ergibt die Ziffer die Rechte pro Triade.

Bedeutung der Rechte bei Files vs. Directories

Symbol	Effekt auf Dateien	Effekt auf Ordner (Directories)
r	Lesen	Inhalt auflisten (<code>ls</code>)
w	Modifizieren	Inhalt ändern (Files anlegen/löschen), sofern x gesetzt ist
x	Ausführen	Betreten / Traversieren (<code>cd</code>)
s / S	setuid/setgid: Wird mit Rechten des Besitzers/Gruppe ausgeführt	setgid: Neue Dateien im Ordner erben die Gruppen-ID des Ordners
t	(Ignoriert)	Sticky bit: Dateien im Ordner dürfen nur von root, File-Owner oder Dir-Owner gelöscht/umbenannt werden

User IDs & Das Setuid-Bit

Jeder Prozess hat drei IDs (sowohl User als auch Group):

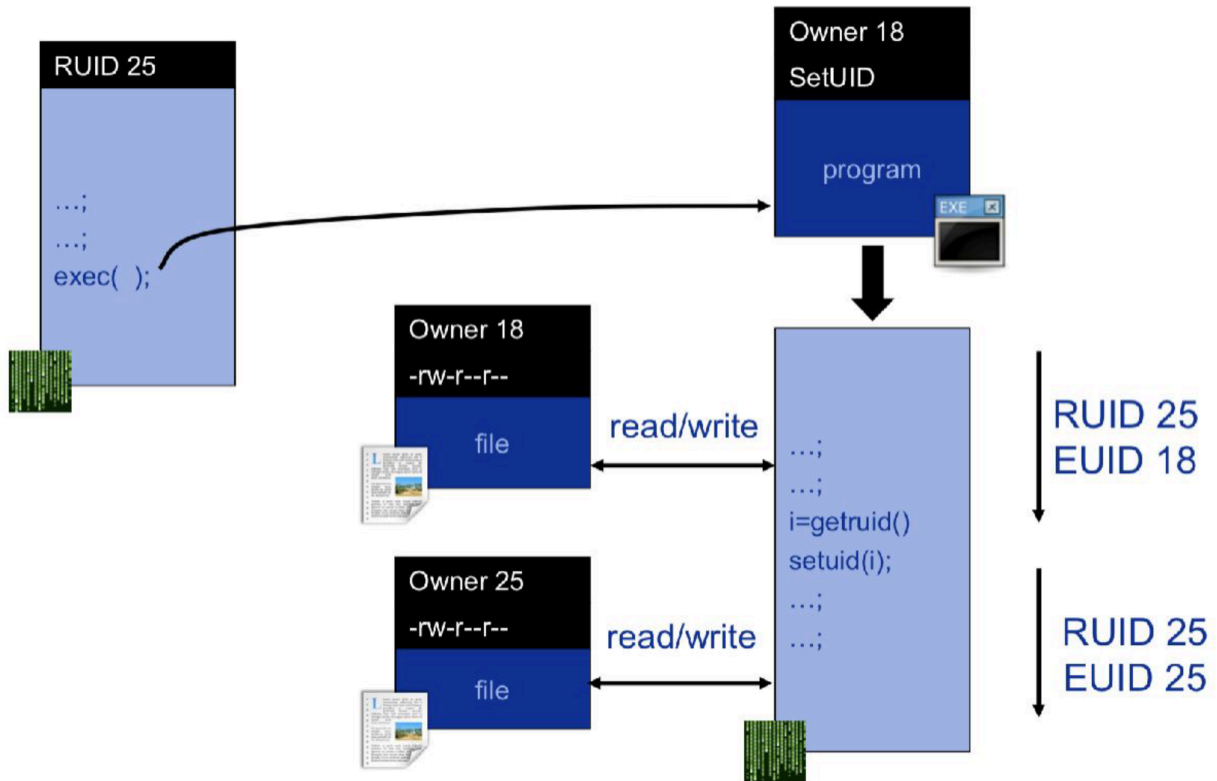
1. **RUID (Real UID):** Wer hat den Prozess gestartet? (Wird vom Parent vererbt).
2. **EUID (Effective UID):** Relevant für **Zugriffskontrollen**. Wird vom Parent geerbt, *außer* das Setuid-Bit ist auf dem Programm gesetzt.
3. **SUID (Saved UID):** Speichert vorherige EUID, falls diese im Betrieb gewechselt wird.

☰ **setuid Beispiel:** `passwd`

`/etc/shadow` (Speichert Passwörter) ist nur von root lesbar/schreibbar (`-rw-----`). Nutzer müssen aber ihre Passwörter ändern. Das Tool `/usr/bin/passwd` hat das setuid-Bit gesetzt (`-rwsr-xr-x 1 root root ...`). Führt ein normaler User es aus, wird die EUID auf 0 (root) gesetzt, wodurch es `shadow` schreiben darf. Die Anwendungslogik verhindert, dass man fremde Passwörter ändert.

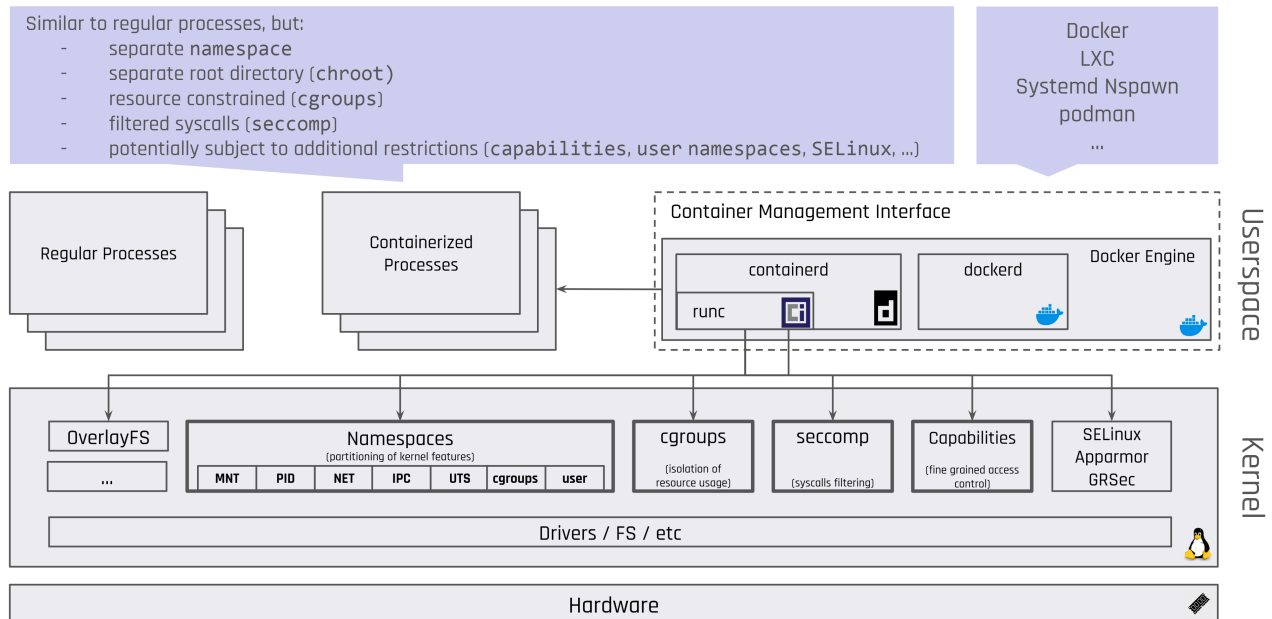
Syscalls zur ID-Änderung:

- Prozesse können mit `seteuid(newId)` etc. ihre Berechtigungen wechseln.
- Privilegierte Prozesse (root) können jede ID annehmen.
- Gibt ein Prozess root-Rechte permanent ab, kann er sie *nicht* zurückholen.



Linux Containers (Leichtgewichtige Virtualisierung)

Das OS vermittelt zwischen Hardware und Userspace-Programmen (über Syscalls). Container nutzen Linux-Kernel-Features, um Prozesse voneinander zu isolieren, als wären sie in eigenen VMs.



1. Namespaces (Namensräume)

Virtualisieren globale Systemressourcen. Wird mit `unshare` gesteuert.

- **PID:** Jeder Container hat seinen eigenen init-Prozess (`PID 1`).
- **IPC:** Eigene Shared-Memory-Segmente.
- **MNT:** Eigene Mount Points (eigenes `/tmp`).
- **NET:** Eigener Netzwerkstack, Routing, Loopback.
- **UTS:** Eigener Hostname/Domainname.
- **user:** Root im Container (`UID 0`) ist draußen ein normaler, unprivilegierter Nutzer.
- **cgroups:** Eigene Ansicht des cgroup-Trees.

2. Chroot ("chroot jail")

Ändert das root-Verzeichnis `/` für den Prozess.

- **Sicherheitsproblem:** Schließt keine offenen File Descriptors! Hatte der Prozess vorher etwas geöffnet, kann er aus dem Jail ausbrechen (root kann generell immer ausbrechen).

3. Cgroups (Control Groups)

Organisieren Prozesse in hierarchischen Gruppen zur **Limitierung und Überwachung von Systemressourcen** (CPU, RAM, Netzwerkbandbreite).

4. Seccomp (Secure Computing)

Schränkt Syscalls massiv ein.

- **Strict Mode:** Nur `read`, `write`, `exit`, `sigreturn` erlaubt. Alles andere killt den Prozess.

- **Filter Mode:** Erlaubte Syscalls über ein BPF (Berkeley Packet Filter) Programm via JSON-Profil definiert. Actions können definiert werden (ALLOW, KILL, ERRNO).

5. Capabilities

Linux hat die root-Privilegien in granulare Einheiten unterteilt. Ersetzt das riskante `setuid` Bit.

- Beispiele: `CAP_SYS_ADMIN` (Mächtigste: mount, unshare), `CAP_NET_ADMIN` (Firewall/IPs konfigurieren), `CAP_SYS_CHROOT`.
- *Probleme bei Docker:* `gdb` geht oft nicht, weil Container standardmäßig `CAP_SYS_PTRACE` nicht haben. *Privileged Mode* gibt dem Container *alle* Capabilities (sehr unsicher!).

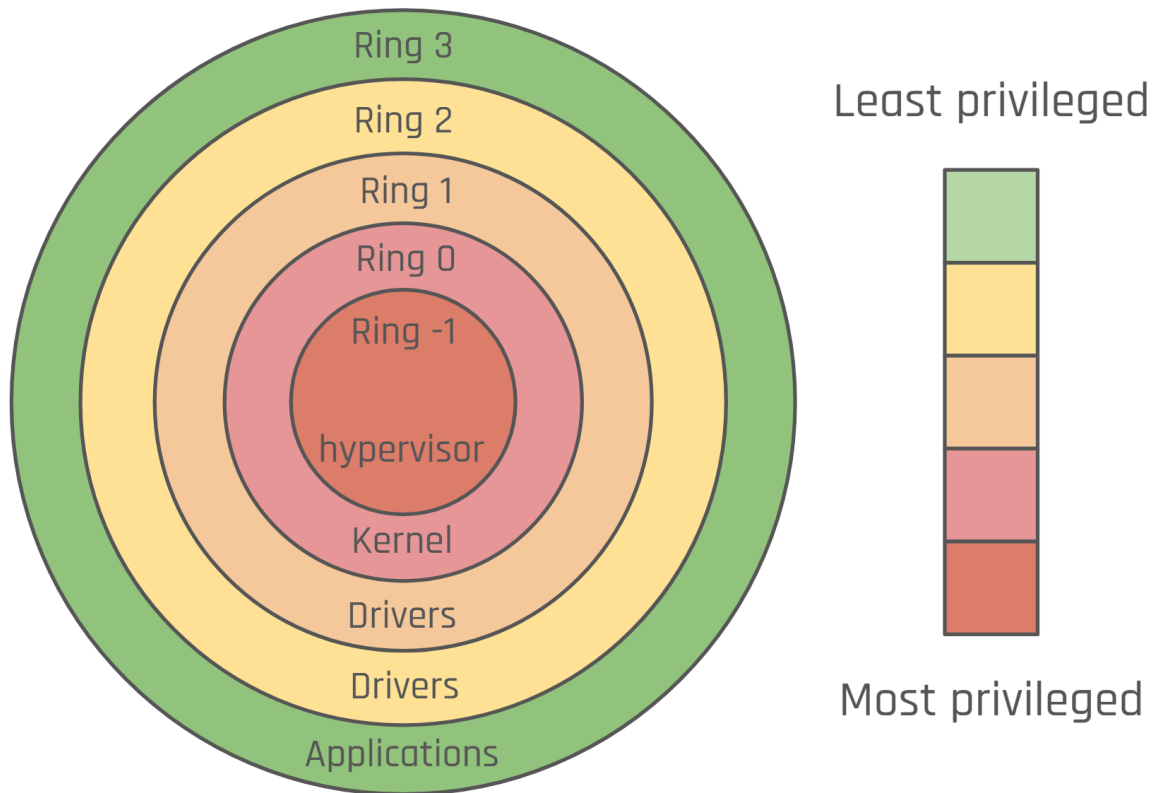
Virtual Machines (Hardware-gestützte Virtualisierung)

Führt unmodifizierte Gast-Betriebssysteme isoliert aus. Der Hypervisor kontrolliert den Zugriff auf die Hardware.

Protection Rings (Ringe)

Hardware-Privilegienlevel (z. B. auf x86).

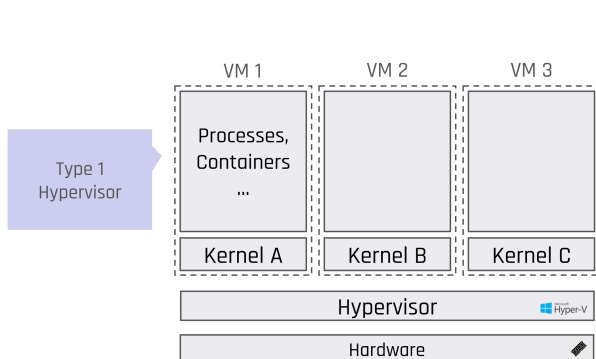
- **Ring 3:** Userspace (am wenigsten privilegiert).
- **Ring 0:** Kernel / Supervisor Mode (Zugriff auf Hardware/CPU Register).
- **Ring -1:** Hypervisor. Nutzt spezielle CPU-Virtualisierungsbefehle (wie Intel VT-X), um Ring-0-Aufrufe der Gast-VMs zu verwalten.



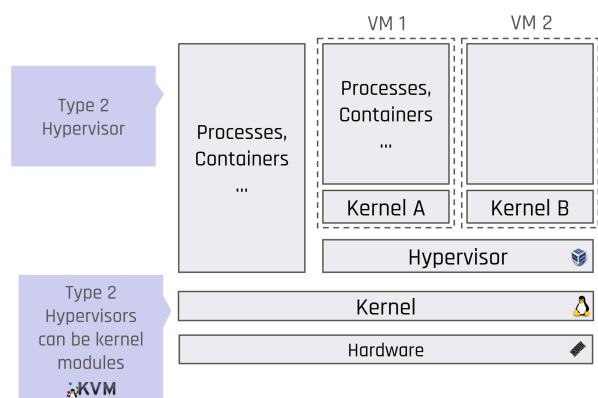
Hypervisor-Typen

1. **Type 1 (Bare-Metal):** Läuft *direkt* auf der Hardware (z. B. Hyper-V).
2. **Type 2 (Hosted):** Läuft als *normaler Prozess / Kernel-Modul* auf einem Host-OS (z. B. KVM).

Also called bare-metal hypervisor, runs directly on the hardware and manages guest OS access.



Also called hosted hypervisor, runs as a regular process on the host



07_Network_Security

Recap of Computer Networks (TCP/IP Suite)

Die Internet Protocol Suite (TCP/IP)

Das Netzwerkmodell ist in 4 Schichten (Layers) unterteilt.

- **Application Layer** (Anwendungsschicht): Behandelt High-Level Protokolle für Netzerkennungen (z.B. HTTP, TLS, DNS).
- **Transport Layer** (Transportschicht): Bietet End-to-End Kommunikationsdienste (z.B. TCP, UDP).
- **Internet Layer** (Vermittlungsschicht): Ermöglicht Kommunikation über unabhängige Netzwerke hinweg (z.B. IPv4, IPv6, ICMP).
- **Link Layer** (Sicherungsschicht): Ermöglicht Kommunikation innerhalb eines lokalen Netzwerks (z.B. Ethernet, Wi-Fi, ARP).

Link Layer & Internet Layer (MAC & IP)

- **Link Layer Protokolle:**
 - Verpackung (Encapsulation) von Paketen der Netzwerkschicht in **Frames**.
 - Zugriffsregulierung auf geteilte Medien (z.B. Kabel, WLAN).
 - Fehlererkennung auf Empfängerseite.
- **MAC-Adressen:** Identifizieren Netzwerkschnittstellen im lokalen Netzwerk eindeutig. Sind 48 Bit lang und werden vom Hersteller zugewiesen.
- **ARP-Protokoll:** Findet die MAC-Adresse zu einer bestimmten IP-Adresse.
- **Internet Layer (IP):** Zuständig für Adressierung, Routing und Zustellung über Netzwerke hinweg.
 - Unzuverlässig: Pakete können verloren gehen, in falscher Reihenfolge ankommen oder Fehler enthalten.
 - **IPv4:** 32 Bit lang. Private Adressen (z.B. `10.0.0.0/8`, `192.168.0.0/16`) können nicht im öffentlichen Internet geroutet werden (benötigen NAT).
 - **IPv6:** 128 Bit lang.
- **Routing Tabellen:** Router verbinden Netzwerke und leiten Pakete weiter. Routing-Tabellen (statisch oder dynamisch, z.B. via OSPF, BGP) beschreiben den "Next Hop" basierend auf der Ziel-IP.

Destination	Netmask	Gateway	Interface	Metric
0.0.0.0	0.0.0.0	203.0.113.1	eth0	1
192.168.1.0	255.255.255.0	0.0.0.0	eth1	0
10.0.0.0	255.255.0.0	192.168.1.254	eth1	2
203.0.113.0	255.255.255.0	0.0.0.0	eth0	1
127.0.0.1	255.0.0.0	0.0.0.0	lo	0

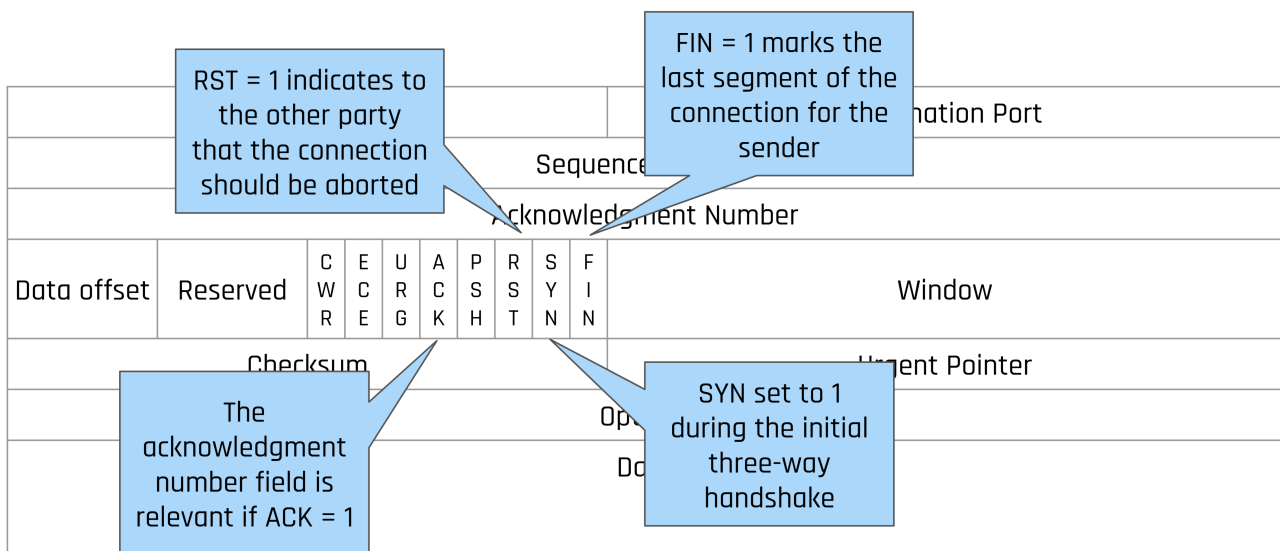
Transport Layer: UDP vs. TCP

Ports

Portnummern identifizieren die kommunizierenden Prozesse (Sender/Empfänger, z.B. Browser mit HTTPS-Server). Größe: 16 Bit (Werte von 0 bis 65535).

- **UDP (User Datagram Protocol):**
 - Verbindungslos, leichtgewichtig, unzuverlässig.
 - Gedacht für Echtzeit-Anwendungen (VoIP, Streaming) oder simple Query-Response (DNS).
 - Fehlerbehandlung muss im Application Layer implementiert werden.
- **TCP (Transmission Control Protocol):**
 - Verbindungsorientiert und zuverlässig.
 - **3-Way Handshake** zum Verbindungsaufbau.
 - Erkennung und Neuübertragung (Re-transmission) von verlorenen oder modifizierten Daten.
 - Sortiert Daten, die in falscher Reihenfolge ankommen.
 - Hosts speichern einen **TCB (Transmission Control Block)** für jede offene Verbindung (speichert IPs, Ports und Sequence-Nummern).

TCP Header & Flags

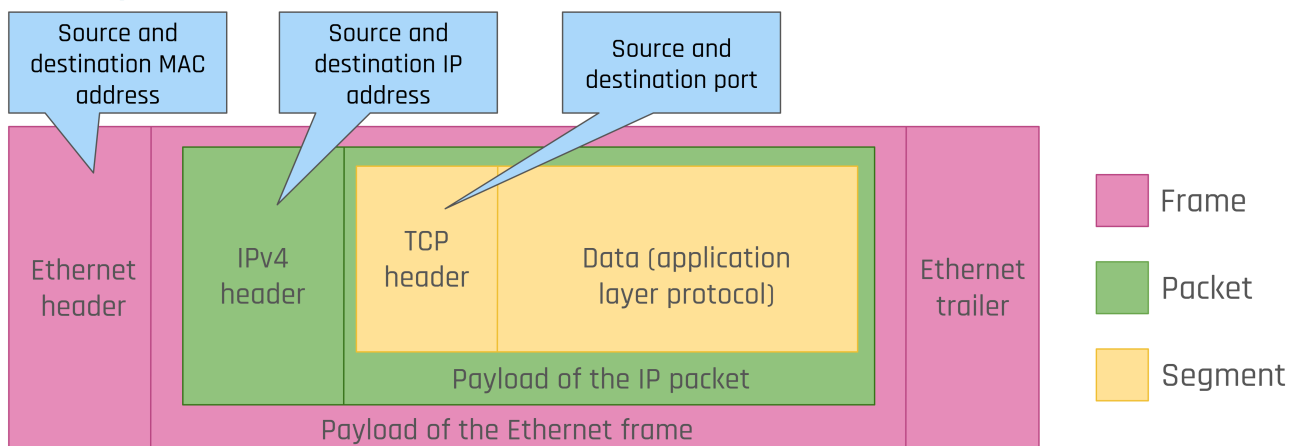


- **SYN**: Auf 1 gesetzt während des 3-Way Handshakes. Zeigt die initiale *Sequence Number* an.
- **ACK**: Bestätigt den Erhalt von Daten. Bestätigt alle Bytes kleiner als die angegebene *Acknowledgment Number*. (Arithmetik erfolgt modulo 2^{32}).
- **FIN**: Markiert das letzte Segment (Sender möchte Verbindung schließen).
- **RST**: Gibt an, dass die Verbindung abgebrochen werden soll.

Encapsulation & Routing Example

🔗 Merkhilfe zur Verkapselung

Data (App) → **Segment** (TCP/Transport) → **Packet** (IP/Internet) → **Frame** (Ethernet/Link).



- Beim **Routing** über mehrere Router ändert sich die Source/Destination **MAC-Adresse** bei jedem Hop (Link Layer), aber die **IP-Adresse** (Network Layer) bleibt von Anfang bis Ende gleich.
- Router entpacken das Frame, lesen die Ziel-IP, bestimmen den Next Hop, verpacken das Paket in ein neues Frame (mit neuer MAC) und senden es weiter.

NAT (Network Address Translation)

- **Source NAT (SNAT)**: Router ersetzt die private Source-IP durch seine öffentliche IP und weist einen neuen Source-Port zu. Mapping wird in einer NAT-Tabelle gespeichert, um Antworten richtig zuzuordnen.
- **Destination NAT (DNAT / Port Forwarding)**: Eingehender Traffic auf einen bestimmten Port der öffentlichen IP wird an einen internen Host im privaten Netz weitergeleitet.

Network Attacks & Spoofing

ARP Spoofing

- **Problem:** ARP bietet keine Authentifizierung; jeder Host im Netz kann antworten. Wer zuerst antwortet, wird meist akzeptiert (Timing kritisch!).
- **Angriff:** Angreifer gibt sich als Default Gateway aus (Man-in-the-Middle). Da Hosts eine ARP Cache Tabelle (IP/MAC Mappings mit einer Time-to-Live / TTL) pflegen, kann diese "vergiftet" werden (ARP Cache Poisoning).
- **Verteidigung:**
 - Statische ARP Einträge (nur für kleine Netze).
 - **Dynamic ARP Inspection (DAI):** Switch-Ports werden als "trusted" (Router) oder "untrusted" (normale Hosts) klassifiziert. Antworten von untrusted Ports werden geblockt, wenn sie nicht mit legitimen Mappings (aus DHCP-Traffic) übereinstimmen.

IP & UDP Spoofing

- **IP Spoofing:** Angreifer sendet IP-Pakete mit der IP des Opfers als Source-Adresse. Meist für DoS oder zum Fälschen von Requests (DNS, TCP-Reset) genutzt.
 - **Gegenmaßnahmen:** Packet Filtering (Ingress/Egress Filtering durch Firewalls/ISPs) und Reverse-path forwarding (Router prüft, ob die Source-IP aus dem Interface kommen kann, auf dem sie empfangen wurde).
- **UDP Spoofing:** Einfach, da verbindungslos. Wird oft für Amplification-Attacks (Verstärkungs-Angriffe) über Protokolle wie NTP oder DNS genutzt (Die Antwort auf den gefälschten Request ist viel größer als der Request selbst und flutet das Opfer).

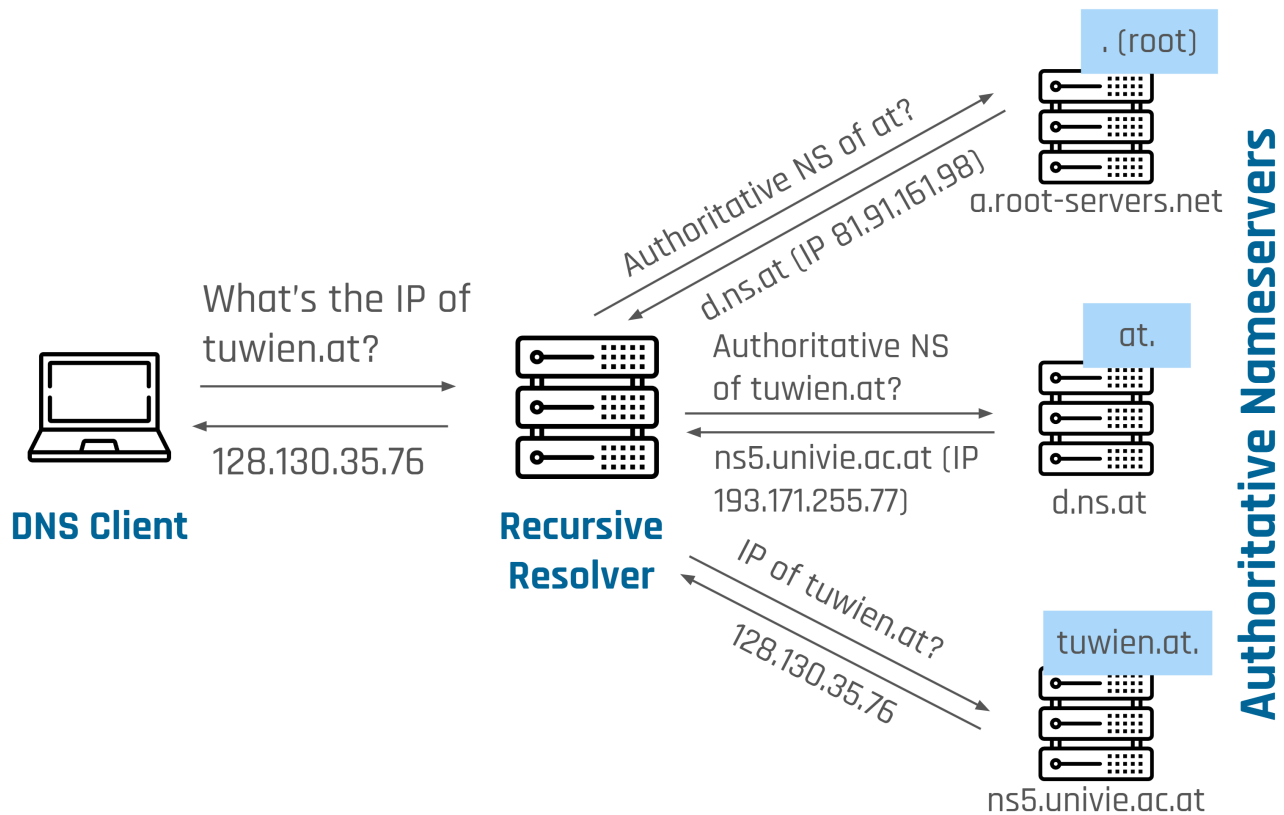
Attacks against TCP

- **SYN Flood (DoS):** Angreifer sendet massenhaft SYN-Pakete, schließt den Handshake aber nie ab. Der Server reserviert für jede "Half-open connection" Ressourcen (TCB) und ist irgendwann ausgelastet.
 - **Verteidigung - SYN Cookies:** Server berechnet die Sequence Number mithilfe einer Hash-Funktion (abhängig von IP, Port und einem Counter $t \bmod 32$). Ein TCB wird erst erstellt, wenn der Client das korrekte ACK zurückschickt.
- **TCP Session Hijacking:** Ziel ist es, Daten in eine etablierte TCP-Verbindung zu injizieren. Der Angreifer muss Ports und Sequence Numbers kennen/erraten. Daten des Angreifers müssen vor denen des echten Senders eintreffen.
- **TCP Reset Attack:** Spezielle Art des Session Hijacking. Angreifer sendet ein TCP Paket mit aktiviertem RST-Flag und gültiger Sequence Number, um die Verbindung abubrechen (wird z.B. von der Great Firewall of China für Zensur genutzt).

DNS Spoofing

DNS Aufbau

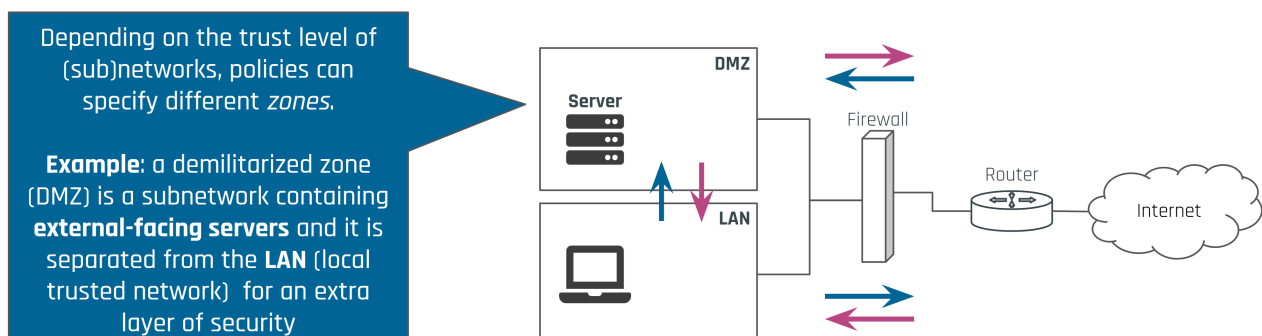
DNS assoziiert Domainnamen mit Infos. **Record Types:** **A** (IPv4), **AAAA** (IPv6), **MX** (Mailserver), **NS** (Nameserver).



- **Angriff:** Angreifer fälscht eine DNS-Antwort mit Fake-Records, um Domains auf bösartige IPs umzuleiten. Wenn der Resolver die Antwort im Cache speichert, sind auch nachfolgende Anfragen betroffen (**DNS Cache Poisoning**).
- **Defenses:**
 - **DNSSEC:** DNS-Einträge werden kryptographisch signiert, um Authentizität/Integrität zu prüfen (Chain of Trust). Bietet keine Vertraulichkeit.
 - **DoH (DNS over HTTPS) / DoT (DNS over TLS):** Verschlüsselt den DNS Traffic.

Firewalls & Netfilter

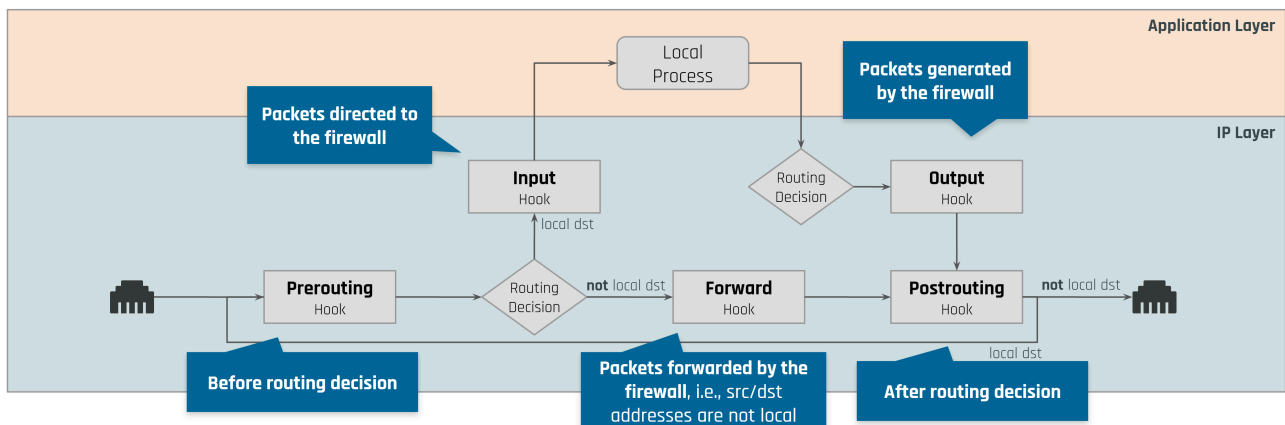
- Monitoren und filtern Traffic basierend auf einer Policy. Trennen trusted (LAN) und untrusted (Internet) Netzwerke.
- **DMZ (Demilitarized Zone):** Subnetzwerk für externe Server, getrennt vom LAN, für extra Sicherheit.



- **Packet Filtering (Stateless):** Regeln werden sequenziell überprüft. Default Drop (sicherer) vs Default Accept. Problem bei stateless: Antworten auf ausgehende Verbindungen müssen explizit erlaubt werden, was Angreifern Tür und Tor öffnet, wenn sie den Source-Port spoofen.
- **Stateful Packet Filtering:** Verfolgt den Verbindungsstatus (State). Kann z.B. erlauben, dass Pakete einer bereits etablierten Verbindung (**ESTABLISHED**) automatisch durchgelassen werden.

Netfilter & iptables (Linux)

- Netfilter definiert **Hooks** (Einhängepunkte) im Linux-Kernel.
- Strukturiert in **Tables** (Art der Aktion: **filter**, **nat**, **mangle**) und **Chains** (Listen von Regeln: **INPUT**, **FORWARD**, **OUTPUT**, **PREROUTING**, **POSTROUTING**).



- **Befehlssyntax:**

```
iptables -t <table_name> -A <chain_name> -s <source_ip> -p <protocol> --dport <port> -j <action>
```

 - **-t** : Table auswählen (z.B. filter)
 - **-A** : Append (ans Ende der Chain hinzufügen)
 - **-P** : Policy setzen (z.B. default drop)
- **Wichtig:** Die Reihenfolge der Regeln ist essenziell (Regeln werden von oben nach unten abgearbeitet).

